ANALISIS PERFORMA ARSITEKTUR PANNs DENGAN PENDEKATAN REPRESENTASI FITUR BERBASIS *RAW WAVEFORM*, *LOG-MEL SPECTROGRAM*, DAN *HYBRID* UNTUK KLASIFIKASI SUARA KEDARURATAN PERKOTAAN

TUGAS AKHIR

Diajukan sebagai syarat menyelesaikan jenjang strata Satu (S-1) di Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut Teknologi Sumatera

Oleh:

Ramon Riping

122140078

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN

2026

DAFTAR ISI

DAFTAR ISI . . . . . **ii**

DAFTAR TABEL . . . . . **iv**

DAFTAR GAMBAR . . . . . **v**

DAFTAR RUMUS . . . . . **vi**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Rumusan Masalah 5

1.3 Tujuan Penelitian 5

1.4 Batasan Masalah 5

1.5 Manfaat Penelitian 6

1.6 Sistematika Penulisan 6

BAB II TINJAUAN PUSTAKA **8**

2.1 Tinjauan Pustaka 8

2.2 Dasar Teori 14

2.2.1 *Environmental Sound Classification* (ESC) 15

2.2.2 Konsep Suara Digital 16

2.2.3 *Dataset* UrbanSound8K 18

2.2.4 Prapemrosesan Data 20

2.2.5 Representasi Fitur Suara 23

2.2.5.1 *Raw Waveform* 23

2.2.5.2 *Log-mel Spectrogram* 25

2.2.6 Augmentasi Data 27

2.2.6.1 *Random Cropping* 27

2.2.7 *Convolutional Neural Network* (CNN) 28

2.2.8 *Pre-trained Audio Neural Networks* (PANNs) 29
2.2.8.1 Res1dNet31 30
2.2.8.2 ResNet38 31
2.2.8.3 Wavegram-Logmel-CNN 33
2.2.9 *Transfer Learning* 35
2.2.9.1 *Freeze Base* 35
2.2.9.2 *Fine-Tuning* 36
2.2.10 Strategi Optimasi dan Skenario Pelatihan 36
2.2.10.1 *Hyperparameter* Pelatihan 37
2.2.10.2 Fungsi Kerugian dan *Cost-Sensitive Learning* 38
2.2.10.3 *Optimizer* dan *Scheduler* 40
2.2.10.4 Mitigasi *Overfitting* (*Early Stopping* dan *Checkpointing*) 41
2.2.11 Metrik Evaluasi 42
2.2.11.1 *Confusion Matrix* 42
2.2.11.2 Metrik Klasifikasi (*Accuracy, Precision, Recall* dan *F1-score*) 44
2.2.11.3 Kurva Evaluasi (*Loss Curve*) 45
2.2.11.4 Uji Signifikansi Statistik (*Wilcoxon Signed-Rank Test*) 46

BAB III METODE PENELITIAN 48
3.1 Alur Penelitian 48
3.2 Pengumpulan dan Prapemrosesan Data 51
3.2.1 Seleksi Kelas 51
3.2.2 Strategi Penyusunan Ulang Data (*Fold Mapping*) 53
3.2.3 Standarisasi Format Audio 54
3.2.4 Augmentasi Temporal (*Cropping*) 54
3.3 Konfigurasi Model 56
3.3.1 Modifikasi Arsitektur PANNs 57

3.3.2 Parameter Konfigurasi *Input* 58
3.4 Eksperimen *Transfer Learning* 58
3.4.1 Lingkungan Implementasi dan Reproduksibilitas 60
3.4.2 Penanganan Ketidakseimbangan Data (*Cost-Sensitive Learning*) 62
3.4.3 Konfigurasi *Optimizer* dan *Scheduler* 63
3.4.4 Strategi Optimasi (*Early Stopping* dan *Checkpoint*) 63
3.4.5 Pemantauan dan Pencatatan Eksperimen 65
3.4.6 Konfigurasi Parameter Pelatihan 66
3.5 Analisis dan Evaluasi 67

DAFTAR PUSTAKA **68**

DAFTAR TABEL

Tabel 2.1 Literatur Penelitian Terdahulu 12
Tabel 2.2 Performa Berbagai Arsitektur PANNs pada *Dataset* AudioSet 29
Tabel 3.1 Konfigurasi Parameter *Input* Audio Model PANNs 58
Tabel 3.2 Spesifikasi Perangkat Keras untuk Pelatihan 60
Tabel 3.3 Daftar Perangkat Lunak dan Pustaka Pendukung 61
Tabel 3.4 Parameter Konfigurasi Pelatihan 66

DAFTAR GAMBAR

Gambar 2.1 *Raw Waveform* 24
Gambar 2.2 *Raw Spectrogram* Hasil STFT 25
Gambar 2.3 *Mel-spectrogram* 26
Gambar 2.4 *Log-mel Spectrogram* 26
Gambar 2.5 Arsitektur Res1dNet31 (diadaptasi dari Kong *et al.* [7]) . 31
Gambar 2.6 Arsitektur ResNet38 (diadaptasi dari Kong *et al.* [7]) . . . 33
Gambar 2.7 Arsitektur Wavegram-Logmel-CNN (diadaptasi dari Kong *et al.* [7]) 34
Gambar 2.8 Ilustrasi *Confusion Matrix* 43
Gambar 2.9 Ilustrasi Dinamika Kurva *Loss* pada Pelatihan Model . . . 46
Gambar 3.1 Diagram Alur Penelitian 48
Gambar 3.2 Ilustrasi Strategi Penyusunan Ulang *Fold* pada *Dataset* UrbanSound8K 53
Gambar 3.3 Ilustrasi Teknik *Random Cropping* 56
Gambar 3.4 Ilustrasi Mekanisme *Checkpointing* 65

DAFTAR RUMUS

Rumus 2.1 Konversi *Stereo* ke *Mono* 20
Rumus 2.2 Operasi *Zero-Padding* 22
Rumus 2.3 *Cross-Entropy Loss* 38
Rumus 2.4 *Penentuan Bobot Cost-Sensitive Learning* 39
Rumus 2.5 *Perhitungan Metrik Accuracy* 44
Rumus 2.6 *Perhitungan Metrik Precision* 44
Rumus 2.7 *Perhitungan Metrik Recall* 44
Rumus 2.8 *Perhitungan Metrik F1-score* 45

BAB I
PENDAHULUAN

1.1 Latar Belakang

Keselamatan berlalu lintas di lingkungan perkotaan merupakan tantangan krusial yang dihadapi masyarakat saat ini, terutama bagi kelompok rentan seperti penyandang disabilitas [1]. Dalam aktivitas tersebut, indra pendengaran berperan sebagai mekanisme deteksi alami yang penting untuk mengetahui kondisi lingkungan di sekitar. Namun, fungsi indra tersebut tidak dimiliki oleh penyandang tunarungu, yang hanya bisa mengandalkan penglihatan mereka untuk memantau keadaan. Ketergantungan penuh pada aspek visual ini dapat menjadi kerentanan serius, mengingat mereka memiliki keterbatasan sudut pandang dan tidak dapat memantau kondisi di luar jangkauan penglihatan. Akibatnya, ancaman yang muncul dari titik buta (*blind spot*), seperti gonggongan dari anjing yang mengejar atau klakson kendaraan yang melaju kencang dari arah belakang, seringkali terlambat disadari akibat tidak terdengarnya peringatan suara. Keterlambatan respon inilah yang secara signifikan meningkatkan risiko terjadinya kecelakaan fatal [2]. Maka dari itu, diperlukan mekanisme alat bantu yang dapat menggantikan peran indra pendengaran dalam mendeteksi ancaman yang muncul dari luar jangkauan visual.

Saat ini, Alat Bantu Dengar (ABD) merupakan perangkat yang umum digunakan untuk menunjang komunikasi verbal penyandang tunarungu. Meskipun efektif untuk komunikasi verbal jarak dekat, alat ini memiliki keterbatasan signifikan dalam konteks keselamatan di luar ruangan. Hal ini disebabkan oleh penurunan selektivitas frekuensi (*reduced frequency selectivity*) yang umum terjadi pada gangguan pendengaran sensorineural, sehingga menyulitkan pemisahan sinyal suara utama dari kebisingan latar belakang yang tumpang tindih [3]. Kondisi ini diperburuk oleh keterbatasan

teknis ABD, di mana sekadar amplifikasi sinyal suara tidak cukup untuk mengembalikan kemampuan pemilahan suara secara alami. Akibatnya, sinyal ancaman penting seringkali tertutup oleh suara-suara lainnya, yang berdampak pada hilangnya kewaspadaan situasional pengguna. Keterbatasan perangkat keras dalam memilah sinyal suara ini memunculkan kebutuhan teknologi bagi penyandang tunarungu agar dapat mengidentifikasi suara bahaya melalui pola sinyal suara, dan bukan sekadar amplifikasi sinyal.

Guna mengatasi keterbatasan ini, dikembangkanlah metode cerdas yang dikenal sebagai klasifikasi suara lingkungan atau *environmental sound classification* (ESC). Integrasi teknologi ini pada alat bantu dengar telah lama diteliti sebagai upaya meningkatkan kesadaran situasi pengguna [4]. Pada tahap awal pengembangannya, sistem ESC umumnya dibangun menggunakan metode *machine learning* konvensional seperti *support vector machine* (SVM) atau *random forest* (RF) [5]. Namun, metode-metode tersebut sangat bergantung pada proses ekstraksi fitur secara manual (*hand-crafted features*) yang kaku, sehingga performanya cenderung menurun drastis ketika dihadapkan dengan kebisingan lingkungan yang bervariatif. Kelemahan metode tersebut memicu pergeseran tren penelitian menuju pendekatan *deep learning*, khususnya *convolutional neural network* (CNN) yang menawarkan kemampuan untuk mempelajari fitur suara dalam bentuk grafis secara otomatis dan hirarkis langsung dari data [6]. Kemampuan adaptasi fitur inilah yang menjadikannya sebagai solusi yang jauh lebih andal dibandingkan metode konvensional. Walaupun menjanjikan akurasi yang lebih tinggi, metode *deep learning* membutuhkan kumpulan data (*dataset*) berskala masif untuk melatih fitur-fitur tersebut secara efektif. Ketergantungan ini menjadi kendala signifikan pada kasus dengan ketersediaan data yang terbatas, sehingga diperlukan strategi pembelajaran khusus agar model tetap memiliki performa yang *robust* (tahan uji).

Sebagai implementasi strategi tersebut, metode *transfer learning* menjadi solusi efektif untuk mengatasi kelangkaan data. Pendekatan ini memanfaatkan

Pre-trained Audio Neural Networks (PANNs) [7], yaitu sebuah kerangka kerja model *deep learning* skala besar yang telah dilatih sebelumnya (*pre-trained*) pada *dataset* AudioSet [8] untuk mengenali berbagai pola suara umum. Salah satu keunggulan PANNs terletak pada variasi arsitektur yang dirancang khusus untuk menangani dua jenis representasi masukan (*input*) audio yang berbeda. Pertama adalah arsitektur berbasis satu dimensi yang mengolah *raw waveform*, yaitu sinyal gelombang suara mentah dalam domain waktu. Kedua adalah arsitektur berbasis dua dimensi yang memanfaatkan *log-mel spectrogram*, yaitu representasi visual yang memetakan intensitas energi frekuensi suara layaknya sebuah citra gambar. Selain itu, PANNs juga menyediakan arsitektur dengan pendekatan *hybrid* yang menggabungkan kedua representasi tersebut, yang secara teoritis berpotensi memaksimalkan akurasi deteksi. Ketersediaan variasi ini memunculkan urgensi untuk mengevaluasi arsitektur mana yang paling optimal untuk diterapkan pada kasus ini, apakah berbasis domain waktu, domain frekuensi, atau penggabungan keduanya (*hybrid*).

Meskipun Kong *et al.* [7] telah memaparkan tolok ukur kinerja model-model tersebut pada *dataset* masif AudioSet [8], performa tersebut belum tentu sebanding ketika diterapkan pada kasus penerapan spesifik dengan ketersediaan data yang terbatas (*data scarcity*), seperti pada kasus klasifikasi suara lingkungan perkotaan. Mayoritas penelitian ESC saat ini cenderung mengandalkan ekstraksi fitur *log-mel spectrogram* karena selektivitas frekuensinya yang menyerupai pendengaran manusia, sekaligus menghindari tingginya kompleksitas komputasi dari representasi sinyal mentah [9], [10]. Namun, proses kompresi spektral tersebut pada *log-mel* juga memiliki kelemahan, yaitu tingginya potensi kehilangan informasi penting seperti transien suara darurat akibat proses reduksi kebisingan latar belakang [11], [12]. Di sisi lain, meskipun pendekatan *hybrid* dapat melengkapi kekurangan setiap fiturnya, besarnya kompleksitas pada arsitektur ini memunculkan dugaan kuat akan tingginya risiko *overfitting* yang lebih tinggi ketika dilatih pada *dataset* yang kecil. Ketidakpastian performa

dan *trade-off* inilah yang menjadi celah penelitian (*research gap*) pertama yang belum terjamah.

Selain ketidakpastian tersebut, terdapat juga perbedaan mendasar pada jenis keluaran klasifikasi model. Arsitektur PANNs dilatih untuk mendeteksi banyak label sekaligus (*multi-label*) [7], sedangkan penelitian ini dirancang untuk memprioritaskan identifikasi sumber bahaya paling dominan (*single-label*). Pendekatan *single-label* ini dipilih untuk menyelaraskan karakteristik *dataset* UrbanSound8K yang memiliki anotasi satu sumber suara dominan (*salient event*) [5]. Meskipun demikian, penelitian ini menyadari bahwa rekaman suara pada *dataset* tersebut maupun di lingkungan perkotaan nyata tidak menjamin kondisi *salient event*, sehingga tumpang tindih (*overlapping*) antar suara sangat mungkin terjadi. Oleh karena itu, sistem pada penelitian ini dibatasi hanya untuk memprediksi dan mengeluarkan keluaran berupa satu kelas suara yang paling dominan berdasarkan probabilitas suara tertinggi. Ketidakmampuan sistem dalam mendeteksi suara sekunder di latar belakang ini disadari dan diakui secara eksplisit sebagai batasan dalam penelitian.

Berdasarkan celah penelitian mengenai *data scarcity* dan penyesuaian jenis keluaran klasifikasi tersebut, evaluasi ulang terhadap adaptabilitas arsitektur PANNs menjadi sangat krusial. Oleh karena itu, penelitian ini bertujuan untuk mengevaluasi dan membandingkan tingkat akurasi serta ketahanan model PANNs menggunakan tiga pendekatan representasi fitur yang berbeda, yaitu *raw waveform*, *log-mel spectrogram*, dan *hybrid* dalam mengklasifikasikan suara tanda bahaya di tengah kebisingan lingkungan perkotaan guna mendukung keselamatan penyandang tunarungu. Studi komparasi ini diposisikan sebagai langkah fundamental untuk menemukan konfigurasi model yang paling *robust* terhadap minimnya data, sekaligus meminimalisir kesalahan deteksi fatal. Dengan demikian, hasil evaluasi ini diharapkan dapat menjadi landasan teknis yang valid bagi pengembangan teknologi asistif yang benar-benar andal untuk menjamin keselamatan komunitas tunarungu.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah:

1. Bagaimana pengaruh perbedaan performa model PANNs dengan representasi fitur *raw waveform*, *log-mel spectrogram*, dan *hybrid* dalam mengklasifikasikan suara bahaya pada kondisi ketersediaan data yang terbatas?
2. Representasi fitur manakah yang menghasilkan performa model paling optimal berdasarkan analisis *confusion matrix*, metrik *F1-score*, dan kurva *loss* untuk meminimalisir kesalahan deteksi pada sistem keselamatan penyandang tunarungu?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah :

1. Menganalisis pengaruh perbedaan performa model PANNs dengan representasi fitur *raw waveform*, *log-mel spectrogram*, dan *hybrid* dalam mengklasifikasikan suara bahaya pada kondisi ketersediaan data yang terbatas.
2. Mengevaluasi pendekatan representasi fitur yang menghasilkan performa model paling optimal berdasarkan analisis *confusion matrix*, metrik *F1-score*, dan kurva *loss* untuk meminimalisir kesalahan deteksi pada sistem keselamatan penyandang tunarungu.

1.4 Batasan Masalah

Batasan masalah yang didefinisikan dalam penelitian ini adalah sebagai berikut :

1. Jenis klasifikasi yang digunakan dalam penelitian ini merupakan klasifikasi *single-label*, sehingga keluaran model berupa kelas suara dengan nilai probabilitas tertinggi daripada kelas lainnya.

2. *Dataset* yang digunakan adalah *dataset* sekunder terstandarisasi, yaitu UrbanSound8K, tanpa melakukan perekaman data primer secara manual.
3. Lingkup klasifikasi dibatasi pada kategori suara lingkungan yang merepresentasikan indikator bahaya atau peringatan bagi keselamatan fisik di jalan raya.
4. Fokus penelitian terbatas pada eksperimen pelatihan (*training*) dan evaluasi performa model *deep learning*, serta tidak mencakup perancangan perangkat keras (*hardware*), pengembangan antarmuka pengguna (*user interface*), maupun implementasi sistem secara *real-time*.

1.5 Manfaat Penelitian

Manfaat dari penelitian ini adalah :

1. Memberikan bukti nyata terkait efektivitas metode *transfer learning* pada arsitektur PANNs serta perbandingan performa antara representasi fitur *raw waveform*, *log-mel spectrogram*, dan *hybrid* dalam mengatasi keterbatasan *dataset*.
2. Berkontribusi dalam pengembangan teknologi asistif berbasis AI yang dapat meningkatkan keselamatan dan kemandirian mobilitas penyandang tunarungu melalui deteksi suara bahaya yang akurat.
3. Menjadi referensi bagi penelitian selanjutnya atau pengembang aplikasi dalam menentukan konfigurasi model yang paling optimal untuk diterapkan pada sistem peringatan dini.

1.6 Sistematika Penulisan

Sistematika penulisan berisi pembahasan apa yang akan ditulis disetiap Bab. Sistematika pada umumnya berupa paragraf yang setiap paragraf mencerminkan bahasan setiap Bab.

Bab I

Bab ini berisikan penjelasan latar belakang dari topik penelitian yang berlangsung, rumusan masalah dari masalah yang dihadapi pada penjelasan di latar belakang, tujuan dari penelitian, batasan dari penelitian, manfaat dari hasil penelitian, dan sistematika penulisan tugas akhir.

Bab II

Bab ini membahas mengenai tinjauan pustaka dari penelitan terdahulu dan dasar teori yang berkaitan dengan penelitian ini.

Bab III

Bab ini berisikan penjelasan alur kerja penelitian, pengumpulan dan prapemrosesan data, konfigurasi model yang digunakan, pengaturan skenario eksperimen, dan metrik analisis serta evaluasi pengujian.

Bab IV

Bab ini membahas hasil implementasi dan pengujian dari penelitian yang dilakukan, serta analisis dan evaluasi yang dapat dipetik dari hasil.

Bab V

Bab ini membahas kesimpulan dari hasil penelitian dan juga saran untuk penelitian selanjutnya.

BAB II
TINJAUAN PUSTAKA

2.1 Tinjauan Pustaka

Penelitian ini dibangun dengan fondasi beberapa penelitian terdahulu yang berfokus pada klasifikasi suara lingkungan dan arsitektur *deep learning*. Berikut merupakan lima penelitian dalam empat tahun terakhir yang memiliki relevansi dengan topik yang diangkat. Ringkasan dari tinjauan pustaka ini juga disajikan pada Tabel 2.1.

Penelitian yang dilakukan oleh Avadhani *et al.* (2024) merupakan penelitian yang dilatarbelakangi oleh tantangan dalam klasifikasi suara di perkotaan dengan karakteristik yang beragam. Dalam pengerjaannya, penelitian ini melakukan ekstraksi fitur manual pada data suara berupa *mel-frequency cepstral coefficients* (MFCC). Fitur tersebut diproses pada arsitektur *artificial neural network* (ANN), *convolutional neural network* (CNN), *recurrent neural network* (RNN), dan *long short-term memory* (LSTM) yang dilatih dari awal dengan kumpulan data (*dataset*) UrbanSound8K. Pemrosesan tersebut menghasilkan keluaran klasifikasi berupa *multi-label*. Penelitian ini berkontribusi memberikan evaluasi kelebihan serta pertimbangan dari setiap model untuk aplikasi nyata di perkotaan. Hasil akurasi yang didapat cukup tinggi pada arsitektur ANN sebesar 94,79%, CNN sebesar 93,64%, dan LSTM sebesar 86,09% pada *dataset* UrbanSound8K. Namun, celah dari penelitian ini adalah tidak adanya mekanisme penanganan ketidakseimbangan antar kelas *dataset* yang dapat merujuk kepada hasil akurasi yang tidak adil, serta penggunaan ekstraksi fitur manual MFCC yang telah melewati proses transformasi yang berpotensi menghilangkan detail pada karakteristik suara [13].

Penelitian selanjutnya dilakukan oleh Mujawar (2025) dalam rangka menganalisis dan membandingkan model *machine learning* yang menawarkan

akurasi dan performa tertinggi dalam klasifikasi kebisingan di perkotaan. Beberapa model yang digunakan dalam penelitian ini adalah CNN, *deep neural networks* (DNN), LSTM, dan *random forest* (RF) yang dilatih dari awal dengan *dataset* UrbanSound8K untuk klasifikasi *multi-label*. Kontribusi dari penelitian ini memberikan literatur mengenai mekanisme penerapan keempat model klasifikasi suara tersebut di lingkungan perkotaan, serta menyarankan arah penelitiannya di masa depan. Keunggulan pada penelitian ini dibuktikan dengan pencapaian akurasi yang memuaskan pada setiap modelnya, dimulai dengan model DNN yang mencapai akurasi sebesar 94,5%, diikuti oleh CNN sebesar 90%, RF sebesar 87%, dan LSTM sebesar 79%. Meski begitu, *dataset* yang digunakan memiliki ketidakseimbangan jumlah data pada setiap kelasnya dan belum ada mekanisme penanganan khusus terkait ketidakseimbangan data tersebut. Dibalik akurasi yang memuaskan, pelatihan model dari awal juga menyebabkan komputasi yang cukup berat sehingga membuka peluang penelitian dengan melakukan *transfer learning*. Peneliti juga menyarankan penelitian terhadap arsitektur model hibrida (*hybrid*) [14].

Menyadari perlunya penerapan *transfer learning* dengan model *pre-trained*, penelitian yang dilakukan oleh Nogueira *et al.* (2022) menerapkan metode tersebut untuk mengidentifikasi suara-suara abnormal dan relevan di lingkungan perkotaan. Penelitian ini menganalisis dan membandingkan performa arsitektur Transformer dengan arsitektur konvensional (*baseline*) maupun CNN untuk klasifikasi suara perkotaan. *Dataset* yang digunakan merupakan *dataset* publik, yaitu UrbanSound8K, ESC-50, dan ESC-10. Untuk evaluasi model *baseline*, peneliti mengekstraksi kombinasi fitur akustik yang kompleks, seperti MFCC dan *chroma*. Namun, pendekatan ekstraksi ini berbeda pada arsitektur Transformer. Model tersebut langsung menerima *input* berupa *spectrogram* dengan memanfaatkan bobot yang telah dilatih sebelumnya (*pre-trained*) pada *dataset* AudioSet, diiringi dengan penerapan augmentasi data. Eksperimen tersebut juga mengusulkan penggunaan *optimizer* Adam dengan *weight decay*

(AdamW) sebagai algoritma optimasi terbaik untuk klasifikasi suara lingkungan. Penelitian ini berhasil menunjukkan performa akurasi klasifikasi yang superior pada model Transformer, yaitu 89,8% untuk *dataset* UrbanSound8K, 95,8% untuk *dataset* ESC-50, dan 99% untuk *dataset* ESC-10. Meskipun menawarkan performa yang sangat kompetitif, model Transformer menuntut kapasitas memori GPU yang cukup berat, yakni mencapai hampir 40 GB VRAM. Selain itu, penerapan teknik augmentasi data suara (seperti *time stretching* dan *pitch shifting*) dalam menangani ketidakseimbangan kelas pada penelitian ini juga berpotensi mendistorsi bahkan menghilangkan karakteristik asli dari sinyal suara [15].

Metode pembelajaran yang sama diterapkan pada penelitian yang dilakukan oleh Trivedi *et al.* (2025), yang terinspirasi dari mekanisme kognitif otak manusia dalam mengidentifikasi suara perkotaan, yang bertujuan untuk menerapkan *deep learning* dengan representasi data yang tepat. Penelitian ini menganalisis dan membandingkan beberapa model CNN, yaitu CNN, CNN-LSTM, dan CNN-YAMnet (model *pre-trained* dengan AudioSet) pada *dataset* UrbanSound8K. Ekstraksi fitur yang digunakan adalah *spectrogram*, MFCC, *linear predictive coding* (LPC), dan dekomposisi *wavelet*. Hasil penelitian ini membuktikan efektivitas integrasi model prapelatihan (*pre-trained*) CNN-YAMNet untuk tugas kategorisasi suara perkotaan dibandingkan dengan arsitektur hibrida konvensional (CNN dan CNN-LSTM). Model CNN-YAMNet berhasil mencapai tingkat akurasi yang sangat mengesankan, yaitu sebesar 98%, membuktikan keunggulan *transfer learning* dalam memproses *dataset* suara yang kompleks. Namun, pemilihan ekstraksi fitur seperti LPC kurang relevan dengan suara perkotaan serta dekomposisi *wavelet* memiliki beban komputasi yang cukup berat. Meskipun hasil akurasi globalnya tinggi, model ini belum mengimplementasikan mekanisme intervensi matematis seperti *cost-sensitive learning* untuk memastikan performa yang adil pada kelas minoritas akibat distribusi data yang tidak seimbang pada UrbanSound8K [16].

Sebagai penutup, penelitian terakhir yang dilakukan oleh Shen (2025) mengangkat masalah terkait kurangnya keragaman skenario pada sumber daya akustik di perkotaan dan penggunaan metode augmentasi umum yang rentan merusak karakteristik suara. Permasalahan tersebut juga didukung oleh pemilihan jaringan saraf tiruan pada penelitian sebelumnya yang tidak interpretabel dan tangguh. Oleh karena itu, metodologi penelitian ini melatih model ResNet-TF dari awal dengan *dataset* yang berisikan kumpulan akustik di perkotaan, yaitu UN15. Dalam kontribusinya, penelitian ini telah menciptakan standar pra-pemrosesan data, pemodelan dalam domain waktu-frekuensi (*spectrogram*), serta augmentasi yang ditambah dengan *screening* sebagai mekanisme penanganan ketidakseimbangan data pada kelas minoritas. Hasil dari penelitian ini berhasil mengintegrasikan pemodelan domain waktu-frekuensi secara grafis yang mampu menangkap detail suara dengan lebih baik pada domain *dataset* suara perkotaan. Akan tetapi, penelitian ini masih bergantung pada pembuatan data buatan (sintetik) dengan melakukan augmentasi dan *screening* untuk kelas minoritas, yang rentan mengubah keaslian suara asli. Pemrosesan tersebut juga memakan waktu yang cukup lama [17].

Berdasarkan kelima literatur di atas, celah penelitian (*research gap*) terlihat dengan jelas. Arsitektur model yang digunakan pada penelitian sebelumnya belum menerapkan pendekatan hibrida (*hybrid*) yang memanfaatkan fitur suara satu dan dua dimensi. Mekanisme penanganan ketidakseimbangan distribusi data pada setiap kelas juga masih belum diterapkan, mengingat *dataset* seperti UrbanSound8K memiliki keterbatasan tersebut. Tidak adanya mekanisme tersebut dapat memengaruhi performa model yang cenderung mendapatkan akurasi tinggi pada kelas yang dominan saja. Adapun penelitian sebelumnya yang menggunakan augmentasi dalam penanganan ketidakseimbangan data juga berpotensi mengubah karakteristik suara, sehingga diperlukan sebuah algoritma yang menangani kendala tersebut tanpa mengubah karakteristiknya. Oleh karena itu, penelitian ini menjawab tantangan beserta celah penelitian

tersebut dengan *transfer learning* menggunakan arsitektur *Pre-trained Audio Neural Networks* (PANNs), yang mengoptimalkan ekstraksi fitur dengan pendekatan hibrida (*raw waveform* dan *log-mel spectrogram*). Penelitian ini juga menerapkan *cost-sensitive learning* untuk menangani ketidakseimbangan kelas tanpa memanipulasi data asli.

Tabel 2.1 Literatur Penelitian Terdahulu

No.	**Judul [Tahun]**	**Masalah & Metode**	**Kontribusi & Keunggulan**	**Keterbatasan**
1.	Multi-Class Urban Sound Classification with Deep Learning Architectures [2024]	Tantangan keberagaman suara perkotaan. Ekstraksi fitur manual MFCC dengan melatih model dasar ANN, CNN, RNN, dan LSTM dari awal pada *dataset* UrbanSound8K.	Mengevaluasi kelebihan model dasar untuk aplikasi nyata. Akurasi tertinggi dicapai oleh ANN (94,79%), CNN (93,64%), dan LSTM (86,09%).	Absennya mekanisme penanganan kelas tidak seimbang, serta ekstraksi MFCC berpotensi menghilangkan detail karakteristik suara asli.

No.	Judul [Tahun]	Masalah & Metode	Kontribusi & Keunggulan	Keterbatasan
2.	Urban Noise Classification Using Machine Learning Techniques: Comparative Analysis and Future [2025]	Pencarian model berakurasi tertinggi dalam klasifikasi suara perkotaan. Melatih CNN, DNN, LSTM, dan RF dari awal dengan *dataset* UrbanSound8K.	Memberikan literatur komparasi performa. Model DNN unggul dengan akurasi 94,5%, diikuti oleh CNN sebesar 90%.	Belum ada penanganan data yang tidak seimbang. Pelatihan dari awal membebani komputasi, serta belum mengeksplorasi *transfer learning* atau arsitektur hibrida.
3.	Transformers for Urban Sound Classification — A Comprehensive Performance Evaluation [2022]	Klasifikasi suara abnormal. *Transfer learning* Transformer (bobot AudioSet) berbasis *spectrogram* dan penerapan augmentasi data.	Memberikan komparasi performa Transformer. Mencapai akurasi 89,8% (UrbanSound8K), 95,8% (ESC-50), dan 99% (ESC-10).	Kebutuhan memori GPU sangat berat (40 GB VRAM), serta augmentasi struktural yang berisiko mendistorsi karakteristik sinyal suara.

No.	Judul [Tahun]	Masalah & Metode	Kontribusi & Keunggulan	Keterbatasan
4.	Deep Learning for Urban Sound Classification: Using CNN and YAMNet Model Integration [2025]	Representasi kognitif untuk klasifikasi. Integrasi model *pre-trained* YAMNet dengan ekstraksi *spectrogram*, MFCC, LPC, dan *wavelet*.	Membuktikan efektivitas CNN-YAMNet dibandingkan arsitektur hibrida konvensional dengan tingkat akurasi yang mengesankan sebesar 98%.	Fitur LPC kurang relevan, beban komputasi *wavelet* berat, serta absennya intervensi *cost-sensitive learning* untuk kelas minoritas.
5.	A Study on AI-Driven Acoustic Feature Design for Urban Noise Classification [2025]	Kurangnya keragaman skenario bising. Melatih ResNet-TF dari awal pada domain waktu-frekuensi dengan *dataset* mandiri (UN15).	Menciptakan standar pra-pemrosesan baru dan augmentasi dengan *screening*. Mampu menangkap detail suara dengan sangat baik.	Sangat bergantung pada data sintetik yang memakan waktu komputasi lama dan tetap rentan mengubah keaslian suara asli kelas minoritas.

2.2 Dasar Teori

Dasar teori ini akan menjadi fondasi utama dalam menyusun metode dalam penelitian ini. Pemaparan diawali dengan membahas mengenai asal usul klasifikasi suara lingkungan (ESC), konsep dasar suara digital, pemrosesan audio digital, serta teknik ekstraksi fitur suara masukan (*input*) yang digunakan untuk menerjemahkan sinyal audio ke dalam format yang dapat diproses oleh mesin.

Selanjutnya, pembahasan akan diarahkan pada inti dari penelitian ini, yaitu arsitektur *deep learning* yang digunakan, mekanisme *transfer learning*, hingga berbagai strategi optimasi dan regulasi pelatihan model. Sebagai penutup, metrik evaluasi akan diuraikan pada bab ini sebagai bahan analisis untuk menyimpulkan hasil dari penelitian ini.

2.2.1 *Environmental Sound Classification* (ESC)

Fenomena yang terjadi di sekitar lingkungan dapat diidentifikasi, baik melalui visual ataupun akustik (suara). Seiring berkembangnya teknologi kecerdasan buatan (AI), komputer (mesin) memiliki kemampuan untuk berperilaku seperti manusia, salah satu contohnya adalah mampu mengidentifikasi suatu kejadian melalui masukan akustik. *Environmental sound classification* (ESC) merupakan klasifikasi kejadian akustik, di mana aktivitas di lingkungan sekitar diidentifikasi dan dikelompokkan sehingga dapat memicu sebuah sistem untuk bekerja (aktif) [18]. Klasifikasi ini memiliki kompleksitas yang lebih rumit dibandingkan dengan jenis klasifikasi akustik lainnya. Pengenalan suara manusia (*speech*) memiliki struktur ejaan dan pengenalan musik memiliki pola ritme dan melodi yang teratur, sedangkan suara lingkungan bersifat tidak statis dan tidak memiliki struktur baku [18]. Hal tersebut menunjukkan bahwa ESC memiliki batasan dan kompleksitas yang khusus.

Penerapan ESC memiliki manfaat dalam beberapa ruang lingkup (domain), salah satunya adalah pengembangan sistem cerdas. Sistem rumah pintar (*smart home*) memanfaatkan tangkapan suara dari mikrofon sebagai masukan dan mengambil tindakan respons atas suara tersebut [19]. Contohnya ketika sistem mendeteksi kaca pecah, maka sistem keamanan akan menyala sebagai respons dari tanda bahaya tersebut. Pada skala ruang yang lebih luas, ESC juga diterapkan pada sistem pemantauan kota pintar (*smart city surveillance*). Teknologi ini membantu infrastruktur kota mendeteksi kejadian darurat di

jalan raya secara otomatis melalui sensor akustik [20]. Dari beberapa bentuk penerapan tersebut, penelitian ini membawa pemanfaatan ESC ke dalam lingkup teknologi alat bantu personal (asistif), yaitu pengenalan kejadian akustik darurat perkotaan khusus bagi penyandang disabilitas tunarungu. Keterbatasan visual penyandang tunarungu dapat dijembatani melalui teknologi pengenalan akustik yang diidentifikasi dari segala sudut pandang [18].

Namun, klasifikasi suara lingkungan memiliki tantangan tersendiri. Tantangan tersebut terletak pada variasi jarak pengambilan sampel dan pencampuran dengan kebisingan latar belakang [18]. Oleh karena itu, diperlukan pendekatan komputasi yang kuat, seperti *deep learning* untuk mengekstraksi pola inti fitur dan mengatasi variasi akustik pada ESC [6].

2.2.2 Konsep Suara Digital

Pada mulanya, suara yang didengar manusia memiliki sinyal fisik yang tidak dapat dilihat secara langsung. Dengan perangkat pengambil suara, seperti mikrofon, sinyal fisik ditangkap dan dikonversikan menjadi sinyal analog dalam bentuk gelombang kontinu. Sistem komputasi tidak dapat memproses gelombang sinyal kontinu, karena arsitektur mesin hanya beroperasi menggunakan data biner (0 dan 1). Untuk memahami sinyal analog tersebut, diperlukan proses pencuplikan gelombang secara berkala untuk diubah menjadi deretan nilai diskrit (angka). Proses konversi ini disebut *analog-to-digital conversion* (ADC) [21]. Hasil dari proses konversi ini berupa deretan sinyal digital yang merepresentasikan nilai diskrit amplitudo gelombang suara pada setiap titik waktu tertentu. Nilai amplitudo inilah yang menjadi fondasi dasar dalam komputasi dan pemrosesan sinyal audio pada mesin.

Adapun tiga komponen utama yang membentuk karakteristik dari suara digital adalah sebagai berikut:

1. ***Sampling Rate* dan Teorema Nyquist**

   Pengambilan seluruh nilai amplitudo tersebut bergantung pada kapasitas

perangkat keras dalam mencuplik sampel pada satu rentang waktu. Konsep ini dilambangkan dengan istilah *sampling rate*, yaitu jumlah titik sampel yang diambil per detik dari sebuah sinyal audio, dengan satuan Hertz (Hz) [21]. Sebagai contoh, jika sebuah audio memiliki *sampling rate* sebesar 32.000 Hz, maka sistem mengambil 32.000 titik sampel amplitudo dalam 1 detik. Semakin tinggi nilai *sampling rate*, maka semakin akurat rekaman digital tersebut merepresentasikan bentuk gelombang suara aslinya.

Terkait hal ini, teorema Nyquist-Shannon mengemukakan aturan matematis bahwa nilai *sampling rate* minimum harus dua kali lipat dari frekuensi tertinggi yang ada pada sinyal suara asli tersebut ($f_s \geq 2f_{max}$) [21]. Aturan ini didasarkan pada konsep frekuensi, yaitu banyaknya gelombang dalam satu detik. Karena satu siklus gelombang penuh harus terdiri dari satu bagian puncak dan satu bagian lembah, Teorema Nyquist memastikan bahwa proses pencuplikan menangkap setidaknya dua titik tersebut agar salah satu bagian gelombang tidak hilang (*aliasing*) saat diubah ke dalam bentuk digital [21].

2. ***Bit Depth***

   Jika *sampling rate* berfokus pada seberapa sering sampel diambil berdasarkan urutan waktu (frekuensi), maka *bit depth* menentukan tingkat ketelitian atau resolusi dari nilai amplitudo pada setiap sampel tersebut. Semakin tinggi nilai *bit depth*, maka semakin halus transisi volume suara yang dihasilkan, sehingga meminimalisir distorsi atau hilangnya informasi sinyal (*quantization error*) [22].

   Rentang nilai diskrit amplitudo ini dikalkulasikan menggunakan persamaan berikut:

   - Total kemungkinan nilai pada resolusi n bit = 2^n
   - Nilai batas bawah (minimum) n bit = $-2^{(n-1)}$
   - Nilai batas atas (maksimum) n bit = $2^{(n-1)} - 1$

Perbedaan rentang antara batas atas dan batas bawah terjadi karena nilai 0 tergolong ke bagian dari nilai positif. Representasi perbedaan ini diatur dalam hukum *two's complement* yang merupakan standar fundamental dalam arsitektur komputasi modern [23].

3. **Format Saluran Suara (*Audio Channel*)**

   Karakteristik spasial dari sebuah rekaman audio ditentukan oleh format saluran suaranya. Format ini bergantung pada jumlah perangkat mikrofon yang digunakan serta posisi pengambilan saat proses perekaman [22]. Secara umum, terdapat dua format saluran suara yang paling dasar, yaitu saluran tunggal (*mono*) dan saluran ganda (*stereo*). Format *mono* merekam seluruh sinyal suara ke dalam satu lintasan tunggal, sedangkan format *stereo* menggunakan dua lintasan terpisah (kiri dan kanan) untuk menciptakan ilusi arah dan dimensi ruang pada suara yang dihasilkan [22].

   Dalam proses komputasi, format saluran ini secara langsung memengaruhi dimensi struktur data pada memori mesin. Suara dengan format *mono* direpresentasikan sebagai satu baris deret (*array*) nilai amplitudo, sedangkan suara *stereo* direpresentasikan sebagai dua baris *array* nilai amplitudo terpisah [22]. Dalam tahap prapemrosesan (*preprocessing*) data untuk pelatihan model kecerdasan buatan (*deep learning*), keberagaman format saluran suara dari data mentah seringkali diseragamkan (*down-mixing*) menjadi format *mono* [7]. Hal ini dilakukan untuk menjaga konsistensi dimensi data *input* pada model dan meminimalkan beban komputasi tanpa menghilangkan fitur utama dari sinyal suara tersebut.

2.2.3 *Dataset* UrbanSound8K

Dataset UrbanSound8K merupakan kumpulan data yang mengandung 8732 suara kejadian di lingkungan perkotaan. Semua *file* audio di dalam UrbanSound8K berdurasi maksimal 4 detik dan berekstensi `.wav` [5].

UrbanSound8K mengelompokkan label datanya ke dalam 10 kelas, di mana kategori tersebut mencakup :

1. *Air conditioner* (pendingin ruangan)
2. *Car horn* (klakson mobil)
3. *Children playing* (anak-anak sedang bermain)
4. *Dog bark* (gonggongan anjing)
5. *Drilling* (aktivitas mengebor)
6. *Engine idling* (mesin menyala statis)
7. *Gun shot* (tembakan senjata)
8. *Jackhammer* (penghancur permukaan keras)
9. *Siren* (sirene)
10. *Street music* (musik jalanan)

Selain *file* audio, *dataset* ini juga menyediakan *file* `.csv` sebagai informasi metadata untuk setiap *file* audionya. Terdapat format penamaan *file* audio di dalam *file* metadata tersebut dengan format : `[fsID]-[classID]-[occurrenceID]-[sliceID].wav`. Penomoran `classID` dimulai dari angka 0 sampai 9, sesuai dengan urutan yang tertera di atas [5].

Semua *file* audio telah disusun ke dalam 10 *fold*, dengan format penamaan folder : `fold1` - `fold10`. Penyusunan ini dibuat sedemikian rupa karena ada beberapa *file* audio yang direkam di tempat dan kejadian yang sama, sehingga beberapa rekaman tersebut disatukan pada *fold* yang sama demi menghindari kebocoran data. Kebocoran data merupakan fenomena yang terjadi saat data yang seharusnya menjadi data uji, tercampur dengan data latih sehingga model mendapatkan karakteristik suara yang sama saat validasi pembelajaran (dan berlaku sebaliknya). Maka dari itu, pembuat *dataset* sangat tidak menganjurkan pengacakan data ulang karena berpotensi mengakibatkan model mengalami *overfitting*, yaitu model memiliki nilai akurasi tinggi hanya saat memprediksi data latihnya saja [5]. Ketika diberikan data baru yang belum pernah dipelajari

sebelumnya, maka model akan gagal memahami data dan mengalami penurunan performa ketika dihadapkan pada suara baru dengan label yang sama.

Dalam *dataset* UrbanSound8K, *file* audionya merupakan hasil kompilasi rekaman dari berbagai kontributor di situs repositori Freesound, sehingga *sampling rate*, *bit depth*, dan format saluran suara tidak seragam [5]. Agar model memiliki dimensi *input* yang sesuai dan seragam, diperlukan tahap prapemrosesan data untuk menyamaratakan dimensi data pada UrbanSound8K.

2.2.4 Prapemrosesan Data

Salah satu kunci utama penentu keberhasilan model kecerdasan buatan dalam tugas klasifikasi adalah kualitas data yang digunakan [24]. Mengingat tingginya variasi karakteristik fisik audio pada *dataset* mentah UrbanSound8K, tahap prapemrosesan (*preprocessing*) data perlu dilakukan sebelum data tersebut digunakan untuk pelatihan model. Tahapan ini bertujuan untuk menyamaratakan karakteristik kualitas data sesuai dengan kebutuhan *input* model untuk mendapatkan hasil performa maksimal [24].

Adapun tahapan prapemrosesan data yang diterapkan pada sinyal suara digital dijelaskan sebagai berikut:

1. ***Down-Mixing***

   Down-mixing merupakan teknik pemrosesan sinyal untuk mereduksi jumlah saluran suara, yang dalam konteks ini melakukan konversi format *stereo* menjadi format *mono*. Pada tugas klasifikasi suara lingkungan, informasi spasial atau arah datangnya suara tidak memiliki relevansi yang signifikan terhadap identitas kelas suara tersebut [7]. Proses reduksi ini dilakukan dengan mengalkulasi nilai rata-rata (*mean*) amplitudo dari saluran kiri dan saluran kanan pada setiap titik sampel waktu yang sama [22]. Persamaan matematis untuk operasi ini dirumuskan sebagai berikut:

$$x_{mono}[n] = \frac{x_{left}[n] + x_{right}[n]}{2}$$

(Rumus 2.1)

Keterangan:

- x : Amplitudo Sinyal
- n : Indeks Waktu
- $x_{mono}[n]$: Amplitudo saluran *mono* pada sampel ke-n.
- $x_{left}[n]$: Amplitudo saluran kiri pada sampel ke-n.
- $x_{right}[n]$: Amplitudo saluran kanan pada sampel ke-n.

Langkah reduksi ini memangkas dimensi struktur data sebesar 50%, yang berdampak langsung pada efisiensi alokasi memori serta meringankan beban komputasi model saat proses pelatihan berlangsung tanpa menghilangkan karakteristik frekuensi utama dari sinyal tersebut [7].

2. ***Resampling***

   Resampling adalah proses transformasi digital untuk mengubah *sampling rate* dari suatu sinyal audio [21]. Dalam konteks tugas klasifikasi, proses ini sangat krusial karena arsitektur model klasifikasi mensyaratkan dimensi *input* yang seragam. Jika *dataset* terdiri dari berbagai rekaman audio dengan *sampling rate* yang bervariasi (misalnya 16.000 Hz, 44.100 Hz, 48.000 Hz), model akan kesulitan memproses panjang *array* yang berbeda-beda untuk durasi rekaman yang sama. Oleh karena itu, seluruh audio mentah tersebut harus diseragamkan ke satu nilai *sampling rate* standar. Demi menjaga karakteristik suara, proses *resampling* harus tetap mematuhi teorema Nyquist-Shannon yang telah diuraikan sebelumnya. Hal ini bertujuan agar frekuensi tertinggi yang merepresentasikan karakteristik utama suara tidak hilang atau mengalami distorsi (*aliasing*) [21].

3. **Penyeragaman Durasi (*Zero-Padding* dan *Truncating*)**

   Karakteristik model klasifikasi pada umumnya mensyaratkan dimensi

input yang bernilai tetap (*fixed-length*) untuk dapat memproses data tersebut [25]. Panjang rekaman audio yang bervariasi menghasilkan panjang *array* sinyal digital yang tidak seragam, sehingga model akan gagal melakukan komputasi jika data mentah tersebut dimasukkan.

Untuk mengatasi inkonsistensi durasi ini, diterapkan teknik *zero-padding* pada sinyal yang berdurasi lebih pendek dari target durasi *input*, dan *truncating* (pemotongan) pada sinyal yang melebihi target. *Zero-padding* adalah metode penambahan deret nilai amplitudo nol, yang berarti keheningan (*silence*) pada bagian ujung akhir *array* sinyal audio [26]. Karena nilai nol merepresentasikan ketiadaan suara, penambahan ini memperpanjang durasi audio tanpa mengubah karakteristik frekuensi maupun informasi dari sinyal aslinya.

Secara matematis, operasi *zero-padding* untuk memperpanjang sinyal audio dirumuskan sebagai berikut [26]:

$$x_{pad}[n] = \begin{cases} x[n], & \text{untuk } 0 \leq n < L \\ 0, & \text{untuk } L \leq n < T \end{cases}$$

(Rumus 2.2)

Keterangan:

- x : Amplitudo Sinyal
- n : Indeks waktu diskret (sampel)
- $x[n]$: Nilai amplitudo pada sampel ke-n
- $x_{pad}[n]$: Nilai amplitudo target pada sampel ke-n
- L : Panjang sinyal awal (jumlah sampel)
- T : Panjang sinyal target (jumlah sampel)

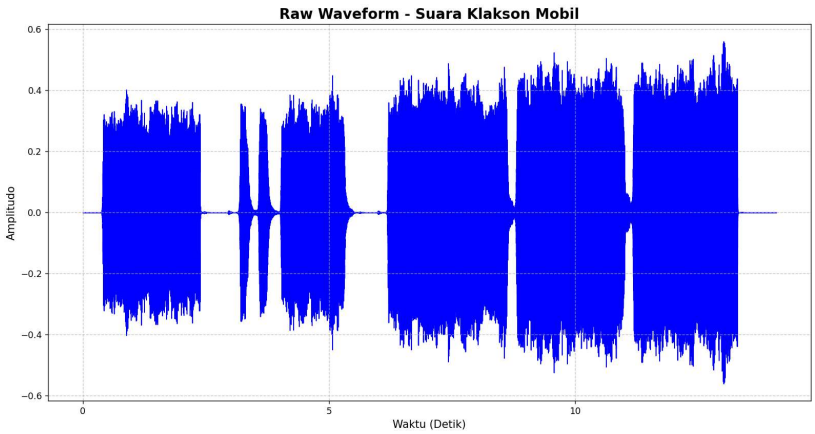
Melalui standarisasi ini, seluruh data audio memiliki dimensi ruang yang seragam sehingga kompatibel dengan persyaratan *input* arsitektur model klasifikasi dan siap untuk diekstraksi fiturnya.

2.2.5 Representasi Fitur Suara

Dalam arsitektur model klasifikasi suara, representasi fitur suara yang dipelajari akan mempengaruhi performa model mengenali karakteristik suara tersebut. Maka dari itu, pembentukan representasi fitur pada suara dapat dilakukan secara terpisah sebagai *input*, atau melalui proses ekstraksi pada lapisan awal model. Representasi fitur suara yang dibahas pada penelitian ini adalah gelombang suara mentah (*raw waveform*) dan spektral *log-mel* (*log-mel spectrogram*).

2.2.5.1 *Raw Waveform*

Raw waveform merupakan representasi digital satu dimensi (1D) yang berisikan deretan nilai diskrit amplitudo pada audio, yang jika divisualisasikan akan membentuk sebuah gelombang audio tersebut. Audio dengan representasi tersebut umumnya memiliki format ekstensi berupa `.wav`, yang berisikan data PCM (*pulse-code modulation*) berupa representasi sinyal digital yang belum terkompresi (*lossless*) [22]. Format tersebut menandakan kualitas serta informasi pada audio tersebut masih murni dan lengkap. Semua *file* audio pada UrbanSound8K memiliki format `.wav`, sehingga keutuhan informasi akustik terjaga pada *dataset* ini [5].



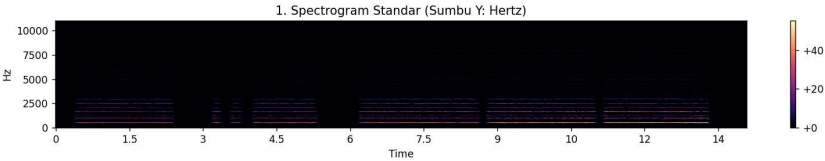
Gambar 2.1 *Raw Waveform*

Proses pembentukan *raw waveform* seperti pada Gambar 2.1 berawal dari gelombang sinyal suara yang sudah mengalami konversi digital, yang kemudian direpresentasikan ke dalam bentuk *array* dengan dimensi berdasarkan indeks durasinya. Total dimensi panjang *array* (N) pada *raw waveform* merupakan hasil kali antara durasi rekaman dalam detik (T) dengan *sampling rate* (f_s). Nilai amplitudo dapat berupa nilai positif bahkan negatif yang umumnya berada pada rentang -1 sampai 1 [26]. Semakin jauh nilai tersebut dari titik nol, maka intensitas suara yang dihasilkan semakin tinggi (kencang).

Dalam domain ESC di lingkungan perkotaan, penggunaan *raw waveform* memungkinkan representasi sinyal yang mempertahankan informasi temporal dan spektral secara utuh, sehingga berpotensi menangkap karakteristik suara berdurasi singkat seperti transien awal. Namun, keutuhan informasi ini juga menjadi tantangan, karena seluruh spektrum kebisingan lingkungan (*noise clutter*) ikut dipertahankan, sehingga model pembelajaran mendalam memerlukan arsitektur ekstraksi fitur yang lebih kompleks untuk membedakan pola yang saling tumpang tindih [27].

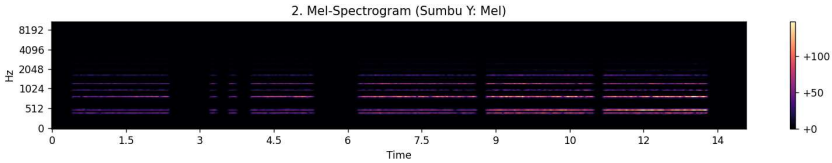
2.2.5.2 *Log-mel Spectrogram*

Spectrogram merupakan representasi grafis fitur suara dalam bentuk domain frekuensi dan waktu. Proses pembentukan *spectrogram* diawali dengan bentuk *raw waveform* yang diproses dengan metode *short-time Fourier transform* (STFT). Metode tersebut dilatarbelakangi oleh masalah mengenai cara dekomposisi setiap frekuensi yang ada pada sebuah gelombang suara. Cara kerja metode STFT adalah dengan memotong audio menjadi bingkai-bingkai waktu yang sangat pendek (milidetik), lalu menerapkan transformasi Fourier pada setiap bingkai secara berurutan untuk mengekstraksi spektrum frekuensinya [28]. Rangkaian spektrum dari setiap bingkai ini kemudian disusun secara temporal membentuk representasi visual dua dimensi. Hasil dari proses STFT dapat dilihat pada Gambar 2.2.



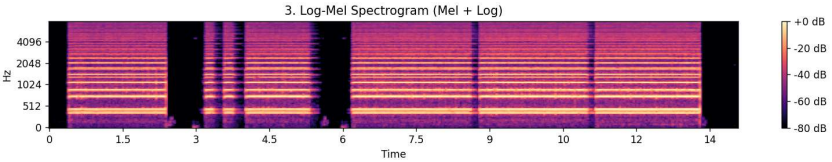
Gambar 2.2 *Raw Spectrogram* Hasil STFT

Setelah mendapatkan gabungan spektrum frekuensi tersebut, skala frekuensi yang digunakan masih berupa skala linear (teratur) dan tidak sesuai dengan pendengaran manusia. Pada kenyataannya, telinga manusia sangat sensitif membedakan nada rendah (misal membedakan 100 Hz dan 200 Hz), tetapi sangat susah membedakan nada tinggi (10.000 Hz dan 10.100 Hz terdengar sama saja). Maka dari itu, skala Mel (*mel-scale*) digunakan untuk mengubah resolusi pembacaan frekuensi suara agar menyerupai persepsi pendengaran manusia, sehingga membuat representasi fitur suara menjadi lebih realistis [28]. *Spectrogram* yang telah diproses dengan *mel-scale* disebut *mel-spectrogram*.



Gambar 2.3 *Mel-spectrogram*

Namun, proyeksi pada *mel-spectrogram* masih memberikan rentang intensitas suara yang cukup jauh, sehingga grafik hanya menampilkan warna terang pada titik intensitas suara yang cukup tinggi. Ini mengakibatkan nilai warna pada detail minor suara yang ingin dipelajari tidak terproyeksi dengan maksimal, sehingga sulit untuk mengklasifikasi dengan detail yang seharusnya penting pada pendengaran manusia. Maka dari itu, operasi logaritmik menjadi solusi dengan mengompresi rentang dinamis (*dynamic range compression*) nilai amplitudo pada *mel-spectrogram*. Seluruh nilai amplitudo pada grafik tersebut dikonversi menggunakan operasi logaritmik ke dalam satuan desibel (dB) [28]. Proyeksi *mel-spectrogram* yang telah diberikan transformasi logaritmik (disebut *log-mel spectrogram*) sangat menyerupai pendengaran manusia sehingga mesin dapat mempelajari suara pada perkotaan layaknya manusia mempelajarinya.



Gambar 2.4 *Log-mel Spectrogram*

Dalam konteks ESC pada lingkungan perkotaan, representasi fitur ini memiliki keunggulan dalam menangkap karakteristik suara yang relevan dengan persepsi pendengaran manusia [28]. Transformasi logaritmik yang diterapkan menghasilkan distribusi energi suara menjadi lebih stabil dan pola frekuensi relevan lebih mudah untuk dipelajari oleh model [7]. Namun, proses transformasi

ini juga menyebabkan hilangnya sebagian informasi asli sinyal, seperti detail fase dan resolusi temporal yang lebih halus, sehingga representasi ini memiliki keterbatasan dalam menangkap karakteristik suara berdurasi sangat singkat secara presisi [28].

2.2.6 Augmentasi Data

Augmentasi data merupakan salah satu teknik yang efektif dalam memperbanyak variasi data dalam ranah pembelajaran mesin (*machine learning*) [25]. Variasi tersebut dilakukan dengan cara memanipulasi karakteristik sinyal suara, seperti menambahkan gangguan (*noise*) atau melakukan pemotongan durasi secara acak. Proses augmentasi dilakukan secara dinamis pada setiap iterasi pelatihan, sehingga model selalu mendapatkan variasi data yang baru. Teknik ini sangat berguna untuk mencegah model menghafal data latih, atau dikenal dengan fenomena *overfitting* [9]. Dengan demikian, model memiliki tingkat generalisasi yang tinggi, yakni kemampuan model untuk memprediksi dengan benar pada data yang belum pernah didapatkan saat pelatihan sebelumnya [25].

Pada penelitian ini, teknik augmentasi yang digunakan berfokus pada ranah temporal, yaitu pemotongan durasi audio secara acak (*random cropping*).

2.2.6.1 *Random Cropping*

Random cropping pada audio adalah teknik memperluas variasi data yang beroperasi pada domain waktu (temporal) dengan cara mengambil potongan data audio dengan durasi yang lebih pendek dari durasi keseluruhan sinyal *input*. Teknik ini diterapkan pada setiap iterasi pelatihan, diawali dengan memilih titik awal pemotongan secara acak (t_0), dengan t_c sebagai target durasi pemotongan. Penempatan titik awal pemotongan diperbolehkan pada rentang durasi $0 \leq t_0 \leq T - t_c$ di mana T merupakan total keseluruhan durasi audio. Pengambilan cuplikan secara acak ini memaksa model klasifikasi untuk tidak

terpaku pada posisi absolut suatu fitur suara, melainkan belajar mengenali pola suara tersebut terlepas dari waktu kemunculannya [9].

2.2.7 *Convolutional Neural Network* (CNN)

Convolutional neural network (CNN) adalah arsitektur model *deep learning* berupa jaringan saraf khusus yang bekerja dengan cara mengenali pola kisi (*grid*) pada data dan meringkas informasi penting yang didapatkan melalui setiap lapisannya. Data yang dikenali oleh jaringan ini dapat berupa data dengan domain waktu (*time-series*) yang bersifat satu dimensi (1D) atau dalam bentuk gambar yang berisikan *grid* atau piksel (2D). Arsitektur ini bekerja dengan memanfaatkan operasi matematika (konvolusi) dengan menggerakkan sebuah *kernel* konvolusi (*filter*) berukuran tertentu ke sebuah gambar, sehingga didapatkan representasi fitur baru yang dikenal sebagai peta fitur (*feature map*) [25].

Arsitektur CNN dibagi menjadi dua bagian besar, yaitu lapisan ekstraksi fitur (*feature extraction layer*) dan lapisan klasifikasi (*classification layer*). *Feature extraction layer* merupakan lapisan yang menerima dan mengekstraksi fitur *input* yang telah diberikan ke dalam peta fitur (*feature map*) multidimensi. Pada tahap akhir ekstraksi, peta fitur tersebut diratakan (*flatten*) menjadi vektor satu dimensi (1D). *Classification layer* akan memproses hasil vektor dari lapisan sebelumnya dan melakukan klasifikasi terhadap fitur-fitur tersebut ke dalam kelas yang telah dipetakan [25].

Feature extraction layer biasanya memiliki beberapa komponen lapisan yang menjalankan aktivitas ekstraksi tersebut. Beberapa komponen tersebut diantaranya adalah *kernel* sebagai jendela pengamat fitur, *stride* yang mengatur langkah *kernel*, *batch normalization* (BN) sebagai pengatur rentang nilai fitur, fungsi *rectified linear unit* (ReLU) yang mengubah nilai negatif menjadi nol, dan *pooling* untuk mereduksi dimensi ukuran fitur. Setelah lapisan ekstraksi, terdapat *classification layer* yang berupa *fully connected layer* (FC *layer*) yang

menghasilkan angka mentah untuk setiap pilihan kelas data. Untuk memperjelas nilai, maka digunakan beberapa fungsi aktivasi untuk mengubah angka mentah tersebut menjadi probabilitas. Dalam kasus klasifikasi *single-label*, digunakan fungsi aktivasi *softmax* dan klasifikasi *multi-label* menggunakan *sigmoid* [25].

2.2.8 *Pre-trained Audio Neural Networks* (PANNs)

Pre-trained Audio Neural Networks (PANNs) menjadi salah satu pilihan yang menyediakan model klasifikasi suara dengan bobot terlatih dari *dataset* suara berskala besar (AudioSet). Arsitektur ini dikembangkan dan dilatih oleh Kong *et al.* (2020) yang menghasilkan keluaran klasifikasi *multi-label* ke dalam 527 kelas suara [7]. Oleh karena itu, diperlukan penyesuaian pada lapisan akhir model agar dapat menghasilkan *output* yang sesuai dengan target kelas suara pada penelitian ini.

Tabel 2.2 Performa Berbagai Arsitektur PANNs pada *Dataset* AudioSet

Arsitektur	**mAP**	**AUC**	**d-prime**
CNN6	0.343	0.965	2.568
CNN10	0.380	0.971	2.678
CNN14	**0.431**	**0.973**	**2.732**
ResNet22	0.430	0.973	0.270
ResNet38	**0.434**	**0.974**	**2.737**
ResNet54	0.429	0.971	2.675
MobileNetV1	**0.389**	**0.970**	**2.653**
MobileNetV2	0.383	0.968	2.624
DaiNet	0.295	0.958	2.437
LeeNet11	0.266	0.953	2.371
LeeNet24	0.336	0.963	2.525
Res1dNet31	**0.365**	**0.958**	**2.444**
Res1dNet51	0.355	0.948	2.295
Wavegram-CNN	0.389	0.968	2.612
Wavegram-Logmel-CNN	**0.439**	**0.973**	**2.720**

Penelitian Kong *et al.* mengenai PANNs juga telah mengevaluasi performa setiap arsitektur menggunakan tiga metrik utama, yaitu *mean average precision* (mAP), *area under the curve* (AUC), dan *d-prime* yang diproyeksikan pada

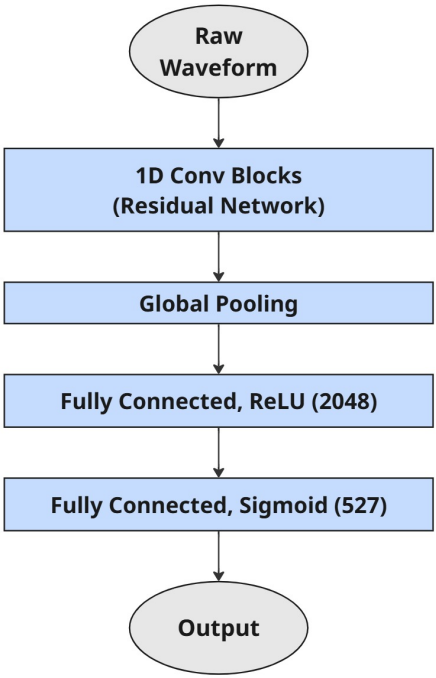
Tabel 2.2 [7]. Semua arsitektur yang tergabung dalam PANNs memiliki representasi *input* yang berbeda-beda. Arsitektur PANNs dengan representasi *input* satu dimensi (1D) meliputi DaiNet, LeeNet, Res1dNet, dan Wavegram-CNN. Selanjutnya, arsitektur dengan representasi *input* dua dimensi (2D) meliputi CNN, ResNet, dan MobileNet. Terakhir, satu-satunya arsitektur dengan pendekatan representasi fitur hibrida (*hybrid*) adalah Wavegram-Logmel-CNN [7]. Dalam penelitian ini, akan diambil 3 model unggul yang mewakili representasi fiturnya masing-masing, yaitu Res1dNet31, ResNet38, dan Wavegram-Logmel-CNN.

2.2.8.1 Res1dNet31

Res1dNet31 merupakan arsitektur ResNet (*residual network*) yang merupakan turunan dari arsitektur CNN berupa satu dimensi milik PANNs yang paling unggul dalam menangani representasi *input* domain waktu dibandingkan model lainnya dengan jenis *input* yang sama [7]. Arsitektur ini menerima *input* berupa sinyal digital satu dimensi (*raw waveform*) dengan *sampling rate* 32 kHz. Keunggulan arsitektur ini adalah kemampuannya dalam mempelajari fitur data suara mentah yang diperoleh secara alami dibandingkan fitur *log-mel spectrogram* yang merupakan *hand-crafted features*.

Proses komputasinya diawali dengan mengekstrak fitur *raw waveform* menggunakan lapisan konvolusi awal dengan *kernel* berukuran 3 untuk memindai pola fitur pada data. Pemindaian tersebut dilengkapi *stride* bernilai 5, sehingga *kernel* berpindah sebanyak 5 sampel untuk pemindaian selanjutnya [7]. Informasi diproses lebih dalam melalui *backbone* (jaringan pengekstraksi fitur utama) ResNet dengan 14 blok residu (blok dengan fitur jalur pintas, atau *skip connection*) [29]. Setiap blok mengandung 2 lapisan konvolusi residu yang masing-masing mengandung dilatasi (perluasan jangkauan *kernel*) 1 dan 2. *Skip connection* merupakan mekanisme yang memungkinkan model untuk melewati beberapa lapisan residu, sehingga fitur yang dihasilkan tidak tereduksi secara berlebihan.

Akhirnya, hasil dari *backbone* dilanjutkan ke 2 lapisan linear terakhir (FC *layer*) untuk mendapatkan kesimpulan klasifikasi final. Seluruh lapisan pada arsitektur ini berjumlah 31 lapisan [7].



Gambar 2.5 Arsitektur Res1dNet31 (diadaptasi dari Kong *et al.* [7])

Gambar 2.5 menunjukkan struktur umum arsitektur Res1dNet31 dalam mengekstraksi fitur dari sinyal *raw waveform*.

2.2.8.2 ResNet38

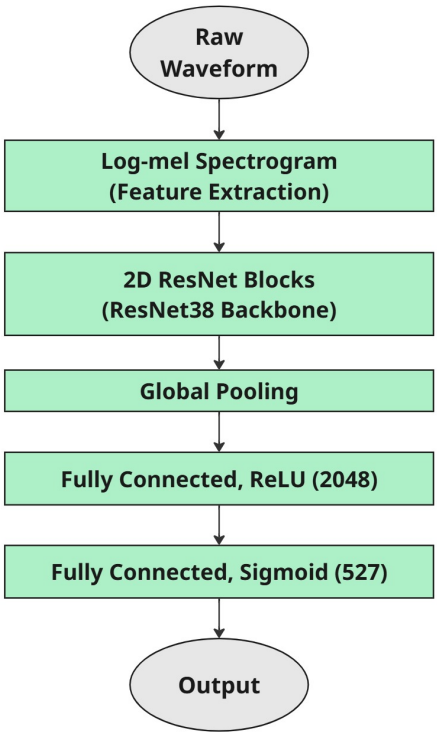
ResNet38 merupakan salah satu arsitektur ResNet dua dimensi milik PANNs dengan nilai mAP tertinggi dalam menangani jenis *input* dengan dimensi tersebut [7]. Berbeda dengan Res1dNet31, arsitektur ini menerima *input* berupa *raw waveform* yang nantinya akan diekstrak dan diproses menjadi

log-mel spectrogram (2D) dengan 1000 *frame* dan 64 *mel bin* oleh ekstraktor fitur [7].

Arsitektur model diawali dengan dua blok konvolusi 64 *filter*[1] berukuran 3×3 dan mengandung BN serta ReLU. Kemudian, *max pooling* berukuran 2×2 diterapkan setelah dua blok tersebut. Selanjutnya, fitur dimasukkan ke dalam *backbone* yang berisikan 16 *basic block*, yang masing-masing berisikan *kernel*, BN, dan ReLU. Kumpulan 16 blok tersebut terdiri dari beberapa kelompok blok berurutan, yaitu 3 *basic block* 64 *filter*, 4 *basic block* 128 *filter*, 6 *basic block* 256 *filter*, dan 3 *basic block* 512 *filter*. Setiap akhir kelompok blok tersebut akan diterapkan *max pooling* berukuran 2×2. Pada akhir *backbone*, diterapkan dua blok berisikan 2048 *filter* berukuran 3×3, BN, dan ReLU, serta diakhiri dengan penerapan *global pooling*. Sebagai lapisan klasifikasi, FC *layer* 2048 *neuron* dengan ReLU diterapkan, dilanjutkan dengan lapisan FC terakhir berisi 527 *neuron* berserta fungsi aktivasi *Sigmoid* sebagai penentu kesimpulan akhir.

Keunggulan arsitektur ini terletak pada kemampuannya dalam menangani *input* domain frekuensi yang sudah tergambar dalam grafis multidimensi. Hal ini dapat memudahkan arsitektur mengenali pola yang sudah diproses sedemikian rupa sehingga menyerupai pendengaran manusia [30].

[1]Pada tabel arsitektur di naskah publikasi asli PANNs, terdapat kesalahan penulisan (*typographical error*) yang mencantumkan angka 512 *filter* pada lapisan awal. Berdasarkan urutan kelogisan arsitektur ResNet serta repositori kode sumber resmi PANNs, jumlah *filter* awal yang akurat adalah 64.



Gambar 2.6 Arsitektur ResNet38 (diadaptasi dari Kong *et al.* [7])

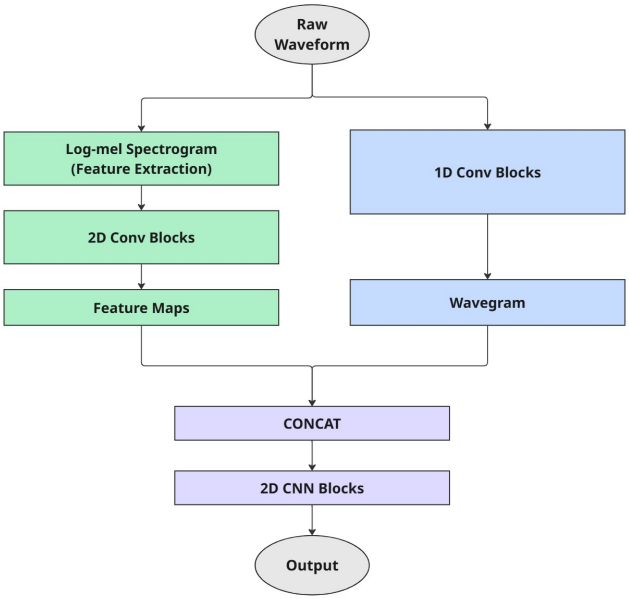
Gambar 2.6 menunjukkan struktur umum arsitektur ResNet38 dengan ekstraktor fitur yang mengekstrak sinyal *raw waveform* menjadi *log-mel spectrogram*.

2.2.8.3 Wavegram-Logmel-CNN

Wavegram-Logmel-CNN merupakan arsitektur dengan pendekatan hibrida (*hybrid*) pada keluarga PANNs yang diperkenalkan oleh Kong *et al.* dalam penelitiannya. Arsitektur ini menerima *input raw waveform* yang diproses dalam dua pendekatan secara bersamaan, yaitu pembentukan *wavegram* dan *log-mel spectrogram* [7].

Untuk membentuk *wavegram*, lapisan awal memproses *input* menggunakan lapisan konvolusi CNN 1 dimensi dengan panjang *kernel* 11 dan *stride* 5. Lalu diikuti dengan 3 blok konvolusi 1 dimensi, di mana setiap blok mengandung 2 lapisan konvolusi residu dengan dilatasi bernilai 1 dan 2. Fitur yang dihasilkan kemudian dibentuk ulang (*reshape*) menjadi sebuah *wavegram*, bersamaan dengan ekstraksi fitur *log-mel spectrogram*. Kedua representasi fitur dua dimensi ini nantinya akan digabungkan dan diproses melalui *backbone* arsitektur CNN14 [7].

Penggabungan kedua pendekatan ini berfungsi untuk menutupi kekurangan masing-masing metode. Sinyal *raw waveform* murni yang kaya akan informasi namun lebih sulit dipelajari oleh model, dikombinasikan dengan *log-mel spectrogram* yang proyeksi polanya lebih mudah dipelajari, meskipun rentan kehilangan sebagian detail informasi akibat proses transformasi matematis [31].



Gambar 2.7 Arsitektur Wavegram-Logmel-CNN (diadaptasi dari Kong *et al.* [7])

Gambar 2.7 menunjukkan struktur umum arsitektur Wavegram-Logmel-CNN dengan penggabungan fitur antara *wavegram* dan *log-mel spectrogram*.

2.2.9 *Transfer Learning*

Transfer learning merupakan metode pembelajaran yang bekerja dengan cara mengadaptasi model yang sudah dilatih (*pre-trained*) dengan konteks umum untuk mempelajari data dengan domain yang lebih spesifik [25], [32]. Metode ini memanfaatkan bobot parameter yang sudah matang dari model sebelumnya, sehingga model tidak perlu dilatih dari nol dengan bobot awal yang masih acak. Pelatihan model dari nol (*training from scratch*) membutuhkan *dataset* berskala besar untuk menyetel jutaan bobot parameter agar dapat memahami fitur secara tepat. Keterbatasan *dataset* dapat menyebabkan *overfitting* akibat model yang menghafal sedikit data, sehingga penerapan teknik ini cukup krusial dalam mengatasi masalah tersebut dengan adanya pengaturan bobot parameter yang hanya perlu sedikit penyesuaian [25].

Dalam kasus penelitian ini, model PANNs yang telah paham mengenali suara lingkungan secara umum diarahkan untuk lebih spesifik mengenali suara kedaruratan di perkotaan. Dalam penerapannya, terdapat dua jenis pendekatan utama dalam melakukan *transfer learning* pada model *pre-trained*, yaitu *freeze base* dan *fine-tuning* [33].

2.2.9.1 *Freeze Base*

Freeze base adalah teknik *transfer learning* yang dilakukan dengan membekukan seluruh bobot parameter pada bagian *backbone* arsitektur model *pre-trained*. Penyesuaian parameter hanya dilakukan pada bagian *head* atau lapisan klasifikasi (*classification layer*) untuk menghubungkan hasil analisis fitur pada kelas target. Pembekuan bobot ini melarang model untuk menyetel bobot parameter saat ekstraksi fitur dengan konteks yang lebih spesifik. Hal tersebut membatasi kemampuan model untuk mempelajari pola fitur yang sangat

spesifik dari data baru, namun memberikan efisiensi komputasi yang sangat tinggi [33].

Dalam kasus *dataset* target yang berjumlah sangat minim, teknik pembekuan bobot ini mempertahankan keandalan bobot parameter yang sudah matang sebelumnya [33]. Jika mekanisme tidak diterapkan, *dataset* minim tersebut dapat merusak setelan parameter bobot sebelumnya dan mengakibatkan fenomena *catastrophic forgetting*, di mana model kehilangan pengetahuan awalnya dan gagal mempelajari konteks data yang baru [34].

2.2.9.2 *Fine-Tuning*

Fine-tuning merupakan tahap lanjutan dari *freeze base*, yang mengizinkan penyesuaian bobot parameter pada lapisan *backbone*. Mekanisme ini dapat memperluas kemampuan model untuk mempelajari data baru yang lebih spesifik, namun beban komputasi meningkat akibat penyesuaian per lapisannya. Untuk mendapatkan hasil yang efektif, diperlukan data dengan jumlah yang cukup agar model memiliki referensi yang memadai dalam memperbarui bobotnya. Selain itu, efektivitas tersebut wajib didukung dengan laju pembelajaran (*learning rate*) model yang sangat kecil untuk penyesuaian bobot yang tidak ekstrem, sehingga pengetahuan awal model tidak rusak [25].

Dalam penerapannya, *fine-tuning* tidak harus dilakukan pada seluruh lapisan *backbone* karena membutuhkan memori yang cukup besar dan rentan mengalami *overfitting*. *Fine-tuning* sering kali dilakukan hanya dengan membuka pembekuan (*unfrozen*) pada beberapa blok konvolusi terakhir saja yang posisinya dekat dengan lapisan klasifikasi [33].

2.2.10 Strategi Optimasi dan Skenario Pelatihan

Dalam dunia pelatihan model, baik dari nol (*training from scratch*) ataupun *transfer learning*, perlu adanya beberapa skenario untuk memastikan model memiliki metode pelatihan yang tepat. Skenario tersebut mengatur mekanisme

jalannya pelatihan. Demi menghasilkan performa model yang optimal, strategi optimasi juga diterapkan dalam proses pelatihan model. Oleh karena itu, tahapan ini mencakup penentuan *hyperparameter*, penerapan fungsi kerugian (*loss function*), penggunaan *optimizer*, hingga teknik mitigasi *overfitting*.

2.2.10.1 *Hyperparameter* Pelatihan

Hyperparameter pelatihan adalah variabel yang nilainya diatur oleh peneliti sebelum pelatihan model dimulai. Pengaturan nilai bertujuan untuk mendapatkan parameter yang paling optimal sehingga menghasilkan performa model terbaik [25]. Terdapat dua *hyperparameter* pelatihan yang paling penting dan mendasar untuk mengatur durasi dan beban komputasi pelatihan, yaitu *epoch* dan *batch size*.

Epoch merupakan satuan pengukuran untuk keseluruhan proses pelatihan model yang menandakan bahwa model telah melihat dan mempelajari keseluruhan *dataset* pelatihan sebanyak satu kali (1 *epoch*). Penentuan jumlah *epoch* merujuk pada proses model yang tidak dapat menghasilkan performa optimal hanya dalam 1 *epoch* akibat bobot model awal yang masih acak. Maka dari itu, model membutuhkan pengulangan *epoch* yang banyak agar fitur inti berhasil disesuaikan dengan bobot parameter. Jumlah *epoch* yang terlalu sedikit menyebabkan model mengalami *underfitting*, yaitu model belum cukup mempelajari data latih. Begitu juga dengan jumlah *epoch* yang terlalu banyak menyebabkan model menghafal data latih (*overfitting*) [25], [35].

Jika *epoch* mengatur durasi pelatihan, maka *batch size* menetapkan banyaknya data (*file*) yang diproses oleh model setiap iterasinya (*step*) dalam 1 *epoch* sebelum bobotnya diperbarui. Penetapan batas jumlah *file* ini diterapkan karena jumlah memori GPU RAM (VRAM) yang bekerja saat pelatihan model terbatas. Memasukkan terlalu banyak *file* pada satu *step* dapat mengakibatkan komputer kehabisan memori (*out of memory*) sehingga pelatihan model dinyatakan gagal [35]. Oleh karena itu, penting untuk mengatur nilai *batch size*

yang tidak hanya mempengaruhi beban komputasi, tetapi sekaligus menetapkan banyak *step* yang dijalankan dalam 1 *epoch*.

2.2.10.2 Fungsi Kerugian dan *Cost-Sensitive Learning*

Nilai kerugian (*loss*) merupakan nilai yang cukup penting dalam evaluasi performa model. Jika nilai akurasi mengukur seberapa banyak model memprediksi label data dengan benar, maka nilai *loss* menunjukkan nilai penyimpangan antara prediksi model dengan label kebenarannya. Nilai *loss* dapat diukur ketika model memprediksi data latih maupun data uji. Maka dari itu, fenomena seperti *overfitting* dapat dideteksi ketika nilai *loss* model saat prediksi data latih (*train loss*) menurun, tetapi nilai *loss* saat prediksi data uji (*validation loss*) meningkat [25].

Untuk mendapatkan nilai *loss*, diperlukan sebuah fungsi yang dapat menghasilkan nilai tersebut (*loss function*). *Cross-entropy loss* adalah salah satu *loss function* yang wajib digunakan dalam kasus klasifikasi data. Fungsi ini menggunakan nilai probabilitas pada keluaran prediksi model yang dirumuskan ke dalam operasi berikut [25]:

$$L = -\sum_{i=1}^{K} y_i \log(p_i)$$

(Rumus 2.3)

Keterangan:

- L : Nilai *loss* yang dihasilkan
- K : Total jumlah kelas
- $\sum$: Penjumlahan denda dari semua kelas.
- y_i : Kunci jawaban kelas ke-i (Nilai 1 untuk kelas yang benar, dan 0 untuk kelas yang salah).
- $\log(p_i)$: Nilai logaritma dari probabilitas tebakan pada kelas ke-i.

Dalam klasifikasi *single-label*, probabilitas seluruh kelas dihasilkan

oleh fungsi *softmax*. Pustaka *deep learning* PyTorch memiliki sebuah fungsi `nn.CrossEntropyLoss()` yang menggabungkan `nn.LogSoftmax()` dan `nn.NLLLoss()` di dalamnya [36]. Penggabungan kedua fungsi tersebut dilakukan demi menjaga stabilitas numerik agar tidak terjadi *error* saat menghitung logaritma dari angka yang sangat kecil. Oleh karena itu, penerapan fungsi *softmax* pada arsitektur model asli diganti dengan lapisan FC biasa, sehingga keluaran dari arsitektur model dibiarkan berupa nilai mentah (*raw logits*) yang akan diproses secara langsung oleh `nn.CrossEntropyLoss()` [25], [36].

Meski begitu, penggunaan fungsi *loss* dapat menghasilkan performa yang kurang valid jika distribusi data setiap kelasnya tidak seimbang. Nilai *loss* yang dihasilkan akan mengatur jalannya pembelajaran model, sehingga model berusaha meminimalkan nilai *loss* pada satu *batch*. Model cenderung berusaha mempelajari kelas dengan jumlah data yang banyak (mayoritas) dibandingkan dengan kelas dengan sedikit data (minoritas) demi nilai *loss* seminimal mungkin. Oleh karena itu, diperlukan sebuah skenario yang menyeimbangkan nilai *loss* pada setiap kelas sesuai kepentingannya [37].

Cost-sensitive learning merupakan modifikasi fungsi *cross-entropy loss* yang menerapkan skenario keseimbangan nilai *loss* dari seluruh kelas [38]. Setiap kelas diberikan sebuah parameter tambahan berupa bobot penalti yang akan dikalikan dengan nilai *loss* dari kelas yang bersangkutan. Dengan begitu, kesalahan prediksi pada kelas minoritas menyebabkan nilai *loss* yang berpotensi lebih tinggi daripada kelas mayoritas, sehingga keseimbangan pembelajaran setiap kelas terjaga. Untuk operasi matematis yang digunakan hanya mengadaptasi penggunaan Rumus 2.3 dengan tambahan bobot pengali (w). Penentuan jumlah bobot pada kelas ke-i (w_i) dirumuskan dalam operasi berikut [39]:

$$w_i = \frac{N_{\text{total}}}{K \times N_i}$$

(Rumus 2.4)

Keterangan:

- N_{total} : Total seluruh data yang digunakan
- K : Total seluruh kelas
- N_i : Jumlah seluruh data pada kelas i.

2.2.10.3 *Optimizer* dan *Scheduler*

Optimizer adalah algoritma yang bertugas memperbarui bobot dan bias pada model berdasarkan nilai *loss* yang didapat. Dalam pengerjaannya, algoritma ini juga memiliki elemen wajib dalam penyesuaian bobot model, yaitu *learning rate*. *Learning rate* (laju pembelajaran) mengatur besar bobot yang diubah dalam satu waktu. Jika pengaturan *learning rate* terlalu besar, maka penyesuaian nilai bobot menjadi ekstrem. Sebaliknya, *learning rate* yang rendah berdampak pada penyesuaian nilai bobot yang pelan, sehingga pelatihan model ikut melambat [25].

Salah satu *optimizer* yang menjadi standar industri *deep learning* saat ini adalah Adam (*adaptive moment estimation*). *Optimizer* ini menggabungkan konsep optimasi dari *optimizer stochastic gradient descent* (SGD) dengan momentum dan RMSProp. SGD dengan momentum memiliki pengetahuan pada langkah yang diambil sebelumnya, sehingga model yang bertemu dengan titik minimum lokal (*local minima*) dapat memanfaatkan akumulasi kecepatan dari langkah sebelumnya untuk keluar dari titik tersebut. Selain itu, Adam juga mengadopsi keunggulan RMSProp dalam mengatur *learning rate* sesuai dengan kondisi gradien, yaitu dengan menurunkan *learning rate* jika nilai gradien tinggi. Penggabungan kedua konsep tersebut yang menjadikan Adam sebagai *optimizer* yang unggul dan adaptif [40].

Di balik keunggulannya tersebut, Adam memiliki kekurangan dalam

penempatan penalti bobot (*weight decay*) yang masih terikat (*coupled*) dengan perhitungan gradien. Penalti tersebut diterapkan agar angka bobot model tidak terlalu besar akibat pengaruh gradien yang terlalu curam, sehingga menghindari *overfitting*. Pada penerapan Adam, penalti ini ditempatkan pada perhitungan langkah bersamaan dengan gradien dan tingkat kecuraman dari RMSProp, yang berdampak pada pengaruh penalti bobot yang sedikit [41].

Oleh karena itu, *optimizer* AdamW (*Adam with decoupled weight decay*) diciptakan sebagai bentuk adaptasi dan pembaruan dari Adam sebelumnya. Adaptasi dilakukan dengan perhitungan penalti bobot yang dilakukan setelah perhitungan nilai langkah dari Adam, sehingga penalti bobot menjadi sangat berpengaruh pada penyesuaian bobot model [41].

Learning rate akhir yang ditentukan oleh *optimizer* seperti AdamW tetap bergantung pada nilai inisialisasi awalnya. Jika model hanya memerlukan sedikit langkah untuk mencapai *global minima* dan nilai *learning rate* awal cukup tinggi, mekanisme *optimizer* tetap berpusat pada nilai *learning rate* awal sehingga membuat konvergensi menjadi tidak stabil. Oleh karena itu, diperlukan sebuah algoritma yang dapat mengatur *learning rate* awal sesuai dengan kondisi tertentu, lebih dikenal dengan *learning rate scheduler*. *Scheduler* memiliki pemenuhan kondisi yang berbeda-beda dalam menyesuaikan *learning rate* [25].

ReduceLROnPlateau merupakan *learning rate scheduler* yang bertugas mereduksi *learning rate* ketika suatu metrik tertentu tidak mencapai titik optimal dalam waktu tertentu. Jika kondisi metrik tidak mencapai titik optimal dalam jumlah *epoch* tertentu (*patience*), maka *learning rate* awal akan dikurangi dengan nilai pengali yang ditentukan. Penerapan tersebut menghasilkan model yang adaptif terhadap hasil performanya [36].

2.2.10.4 Mitigasi *Overfitting* (*Early Stopping* dan *Checkpointing*)

Overfitting merupakan kondisi ketika model menghafal data latih, sehingga model gagal mempelajari inti fitur dan gagal diterapkan pada data di luar pelatihan.

Fenomena ini disebabkan oleh beberapa faktor, salah satunya adalah penetapan jumlah *epoch* yang berlebihan sehingga model yang seharusnya telah berhasil menangkap inti fitur pada *epoch* tertentu malah mempelajari data yang sama berulang kali. Akan tetapi, jumlah *epoch* yang terlalu sedikit dapat menyebabkan model gagal mencapai performa terbaiknya akibat kurangnya waktu belajar (*underfitting*). Oleh karena itu, algoritma pemberhentian pelatihan pada *epoch* tertentu diperlukan untuk menjaga performa optimalnya [25].

Early stopping merupakan teknik mitigasi yang menghentikan pelatihan lebih awal daripada *epoch* maksimal yang ditetapkan. Teknik ini mengambil sebuah metrik tertentu dan menetapkan batas toleransi (*patience*) ketika metrik tersebut tidak melampaui titik optimal sebelumnya. Jika *patience* telah tercapai, pelatihan model akan diberhentikan secara otomatis sehingga titik optimal sebelumnya dijadikan nilai performa terbaiknya [42].

Untuk menyimpan bobot model dengan performa terbaik, penerapan *early stopping* juga didukung dengan sistem *checkpointing*. *Checkpointing* adalah sistem yang menyimpan bobot parameter model saat mencapai nilai metrik performa tertingginya. Jika performa terbaik model pada jejak *epoch* sebelumnya terlampaui, maka sistem akan menyimpan kembali bobot model terkini [25].

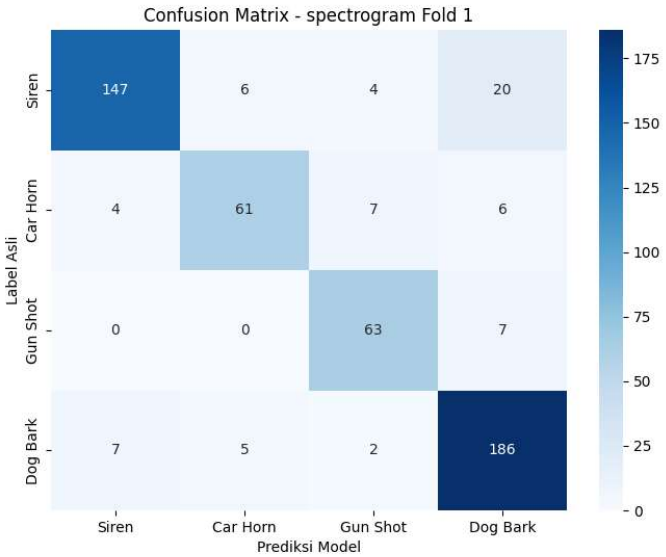
2.2.11 Metrik Evaluasi

Metrik evaluasi merupakan komponen tolok ukur yang menjadi nilai performa model dalam menjalankan tugasnya. Dalam model klasifikasi, metrik tersebut mengukur banyak aspek, yakni jumlah kebenaran hasil prediksi model, akurasi model, hingga dinamika pergerakan grafik *loss* selama proses pelatihan.

2.2.11.1 *Confusion Matrix*

Confusion matrix adalah metrik yang memproyeksikan pemetaan jumlah prediksi kelas benar dan salah model pada data uji. Pemetaan tersebut disusun ke dalam tabel dengan jumlah baris dan kolom yang sesuai dengan jumlah kelas

data. Metrik ini membandingkan hasil kelas prediksi model pada data dengan kelas kebenaran pada data tersebut [43]. Elemen-elemen hasil evaluasi pada metrik ini dikategorikan ke dalam empat komponen utama, yaitu:



Gambar 2.8 Ilustrasi *Confusion Matrix*

1. *True Positive* (TP)

   Model dengan tepat memprediksi data kelas aktual X sebagai kelas X.

2. *True Negative* (TN)

   Model dengan tepat memprediksi data kelas lain (Y) bukanlah kelas X.

3. *False Positive* (FP)

   Model keliru memprediksi data dari kelas lain (Y) sebagai kelas X.

4. *False Negative* (FN)

   Model keliru memprediksi data kelas aktual X sebagai kelas lain (Y).

2.2.11.2 Metrik Klasifikasi (*Accuracy, Precision, Recall* dan *F1-score*)

Evaluasi model klasifikasi umumnya menggunakan sebuah kelompok metrik klasifikasi. Kelompok metrik tersebut menggunakan nilai keempat kategori yang digunakan pada *confusion matrix* sebelumnya, yakni *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Keempat metrik tersebut terdiri dari *accuracy*, *precision*, *recall*, serta *F1-score* [43].

Accuracy merupakan metrik yang mengukur rasio prediksi model yang benar (baik TP maupun TN) dari seluruh prediksi tersebut. Perhitungan metrik tersebut dirumuskan ke dalam operasi berikut [43]:

$$Accuracy = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$$

(Rumus 2.5)

Precision mengkalkulasikan rasio prediksi benar model pada suatu kelas dari seluruh data yang diprediksi model dengan kelas tersebut. Pengukuran dirumuskan ke dalam rumus berikut [44]:

$$Precision = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

(Rumus 2.6)

Recall mengukur kemampuan model dalam memprediksi seluruh data yang secara aktual benar pada kelas tersebut. Pengukuran dirumuskan ke dalam rumus berikut [44]:

$$Recall = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

(Rumus 2.7)

F1-score adalah nilai rata-rata harmonis dari kedua metrik sebelumnya, yaitu *precision* dan *recall*. *Precision* yang berfokus pada banyaknya prediksi model kelas tertentu bertolak belakang dengan *recall* yang berfokus pada

seberapa benar model saat data dengan kelas tersebut dipanggil. Oleh karena itu, *F1-score* menjadi nilai keseimbangan antara kedua metrik ini, sehingga tingginya *F1-score* menandakan keunggulan model dalam tugas klasifikasinya [44]. Perhitungan rata-rata harmonis dalam *F1-score* dirumuskan sebagai berikut:

$$F1\text{-}score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

(Rumus 2.8)

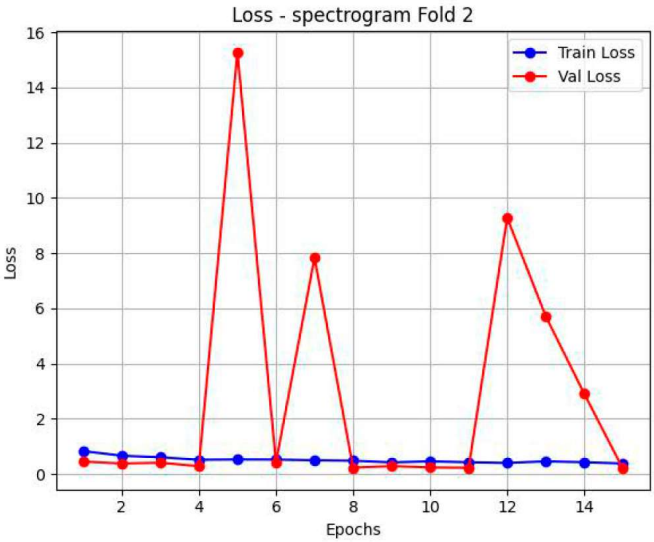
Dalam kasus klasifikasi multikelas (lebih dari dua kelas), perhitungan *F1-score* dapat diperluas menjadi tiga perhitungan utama untuk mengakomodasi distribusi data pada setiap kelasnya [44]. Ketiga varian tersebut adalah *micro F1-score*, *weighted F1-score*, dan *macro F1-score*.

Micro F1-score menghitung metrik dengan menjumlahkan seluruh TP, FP, dan FN dari semua kelas, sehingga perhitungan ini rentan didominasi oleh kelas mayoritas. Berbeda dengan *weighted F1-score* yang menghitung *F1-score* untuk setiap kelas secara terpisah, lalu dikalikan dengan bobot persentase jumlah data pada masing-masing kelas tersebut dan dicari nilai rata-ratanya. Terakhir, *macro F1-score* menghitung *F1-score* untuk setiap kelas secara terpisah, lalu mencari nilai rata-ratanya secara adil tanpa mempedulikan jumlah data pada kelas tersebut (*unweighted mean*). Varian ini memaksa model untuk mengevaluasi kelas minoritas sama pentingnya dengan kelas mayoritas, sehingga sangat ideal untuk mengukur performa model pada *dataset* yang tidak seimbang (*imbalanced data*) [44].

2.2.11.3 Kurva Evaluasi (*Loss Curve*)

Kurva evaluasi *loss* merupakan metrik pengukur tingkat kesalahan model dalam memprediksi data latih (*train loss*) maupun data uji (*validation loss*). Pembacaan kedua kurva (train dan validation) loss tersebut penting dalam

menganalisis apakah model tersebut mempelajari atau sekedar menghafal data [25]. Fenomena *overfitting* digambarkan dengan kurva *train loss* yang menurun, tetapi kurva *validation loss* yang meningkat. Sedangkan *underfitting* digambarkan dengan kurva *loss* keduanya yang cukup tinggi dan sulit untuk turun [25].



Gambar 2.9 Ilustrasi Dinamika Kurva *Loss* pada Pelatihan Model

2.2.11.4 Uji Signifikansi Statistik (*Wilcoxon Signed-Rank Test*)

Dalam evaluasi model *machine learning*, perbedaan nilai rata-rata pada hasil akhir performa model tidak dapat secara langsung menyimpulkan bahwa satu model lebih unggul daripada model lainnya. Perbedaan nilai tersebut dapat terjadi secara kebetulan disebabkan oleh variansi pembagian data uji. Oleh karena itu, diperlukan sebuah metode uji signifikansi statistik yang bertujuan untuk memvalidasi perbedaan signifikansi tersebut secara ilmiah [45].

Wilcoxon signed-rank test merupakan uji hipotesis statistik non-parametrik yang digunakan untuk membandingkan dua nilai sampel yang berpasangan

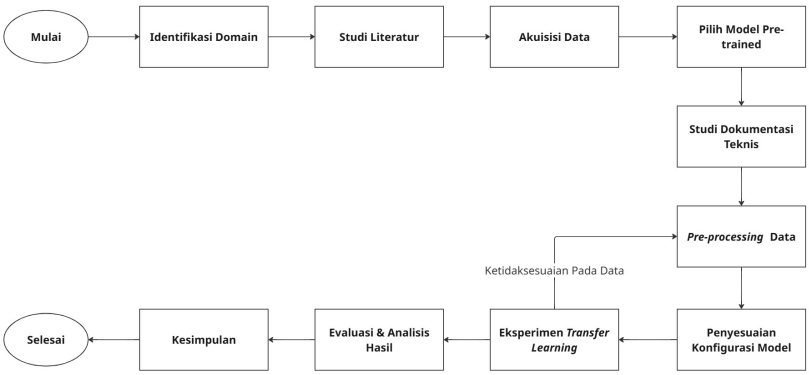
(*paired samples*) dalam menilai perbedaan rata-rata peringkat (*mean ranks*) yang signifikan di antara keduanya. Dalam konteks perbandingan model klasifikasi dengan *k-fold cross validation*, metode uji ini beroperasi pada selisih nilai performa (seperti *F1-score*) dari kedua model pada setiap lipatan (*fold*) yang bersesuaian [45].

Penggunaan uji non-parametrik ini sangat direkomendasikan karena hasil performa algoritma *machine learning* pada beberapa *fold* data umumnya tidak terdistribusi secara normal, sehingga penggunaan uji parametrik standar seperti *t-test* menjadi tidak valid [45]. Jika hasil pengujian menghasilkan nilai probabilitas (*p-value*) di bawah ambang batas signifikansi yang ditetapkan (umumnya $\alpha = 0.05$), maka hipotesis nol (*null hypothesis*) ditolak. Penolakan tersebut menjadi bukti matematis bahwa perbedaan performa antara kedua model yang diuji diakui signifikan secara statistik, bukan karena faktor kebetulan [45].

BAB III
METODE PENELITIAN

3.1 Alur Penelitian

Penelitian ini dilaksanakan melalui serangkaian tahapan sistematis guna memastikan model klasifikasi suara ancaman bahaya yang dikembangkan dapat bekerja optimal pada domain keselamatan penyandang tunarungu. Diagram alur penelitian ditunjukkan pada Gambar 3.1.



Gambar 3.1 Diagram Alur Penelitian

Penjelasan rinci mengenai tahapan penelitian adalah sebagai berikut:

1. **Identifikasi Domain Penelitian**

   Tahap awal dilakukan dengan mencari referensi domain penelitian untuk klasifikasi suara. Berdasarkan pertimbangan ketersediaan kumpulan data (*dataset*) dan urgensi model klasifikasi, domain yang dipilih adalah domain keselamatan publik yang difokuskan untuk alat bantu penyandang tunarungu.

2. **Studi Literatur**

   Dilakukan kajian literatur mendalam untuk menemukan urgensi penelitian,

khususnya mengenai kebutuhan teknologi asistif yang mampu mengurangi risiko kecelakaan bagi penyandang tunarungu melalui pengenalan sinyal bahaya. Ini menjadi landasan teoritis bagaimana teknologi klasifikasi suara dapat menjadi solusi yang efektif dalam mengatasi permasalahan tersebut.

3. **Identifikasi dan Akuisisi Data**

   Akuisisi data dilakukan dengan mengambil *dataset* sekunder yang sudah terstandarisasi, yaitu UrbanSound8K [5]. Akuisisi data primer tidak dilakukan demi menghindari *device bias* akibat perekaman *dataset* dengan perangkat non-standar (seperti smartphone) serta kendala regulasi keamanan pada perekaman kelas *gun_shot*. Dari *dataset* tersebut, dilakukan seleksi kelas sinyal bahaya dengan strategi antisipasi terhadap kendala ketidakseimbangan data (*imbalanced data*) yang telah disiapkan sejak awal.

4. **Pemilihan Model *Pre-trained***

   Mengingat keterbatasan data berisiko menyebabkan kegagalan *training from scratch*, digunakan pendekatan *transfer learning* dengan memanfaatkan model *pre-trained*. Tiga varian arsitektur PANNs (*Pre-trained Audio Neural Networks*) [7] dipilih untuk mewakili pendekatan representasi fitur yang berbeda, yaitu Res1dNet31 (domain waktu), ResNet38 (domain frekuensi), dan Wavegram-Logmel-CNN (*hibrida*). Arsitektur PANNs dipilih karena telah dilatih pada *dataset* masif (AudioSet) dan memiliki performa yang teruji.

5. **Studi Dokumentasi Teknis**

   Tahap ini mempelajari karakteristik data dan model yang digunakan. Fokus utama dalam tahap ini adalah memahami mekanisme pembagian data (*fold*) pada *dataset* guna mencegah kebocoran data (*data leakage*) dan mempelajari variasi representasi fitur pada arsitektur PANNs untuk menentukan skenario komparasi yang tepat.

6. ***Pre-processing* Data**

   Serangkaian proses dilakukan untuk mengubah data mentah menjadi format yang siap latih, meliputi penanganan struktur *fold*, seleksi kelas, dan penyesuaian format audio, serta penerapan teknik augmentasi temporal berupa *random cropping* untuk menstandarisasi durasi *input*.

7. **Penyesuaian Konfigurasi Model**

   Dilakukan modifikasi pada arsitektur model agar sesuai dengan tujuan klasifikasi 4 kelas bahaya, serta penerapan strategi untuk menangani ketidakseimbangan data.

8. **Eksperimen *Transfer Learning***

   Tahapan ini menjadi tahapan inti di mana model dilatih ulang (*fine-tuning*) untuk mengenali karakteristik suara spesifik. Proses ini mencakup eksperimen *hyperparameter* dan pelatihan ketiga variasi model untuk menemukan konfigurasi parameter pelatihan yang paling optimal. Proses ini bersifat iteratif. Apabila performa model belum mencapai target yang diharapkan, maka akan dilakukan penyesuaian ulang pada tahap *pre-processing* data untuk melakukan penyesuaian data dan melakukan pelatihan kembali hingga diperoleh model yang optimal.

9. **Evaluasi dan Analisis Hasil**

   Performa model hasil *fine-tuning* dievaluasi menggunakan metrik *F1-score*, nilai *loss*, dan analisis *confusion matrix*. Evaluasi ini bertujuan untuk membandingkan dan menganalisis performa antara ketiga model dalam melakukan klasifikasi suara ancaman bahaya.

10. **Kesimpulan**

    Berdasarkan hasil evaluasi, dilakukan perbandingan komparatif untuk menyimpulkan model mana yang memiliki performa paling unggul dan stabil.

3.2 Pengumpulan dan Prapemrosesan Data

Sumber data utama dalam penelitian ini adalah *dataset* publik **UrbanSound8K** [5]. *Dataset* ini diunduh dalam format terkompresi (`.zip`). Setelah diekstraksi, struktur *dataset* terdiri dari 10 folder (masing-masing mewakili satu *fold*) beserta satu *file metadata* (`.csv`) yang memuat informasi nama *file* audio, *class ID*, dan *fold* asal.

Namun, *dataset* mentah ini memerlukan serangkaian penyesuaian agar relevan dengan konteks keselamatan penyandang tunarungu. Mengingat *dataset* ini awalnya berisi 10 kelas suara umum di perkotaan yang tersebar dalam 10 *fold*, penelitian ini memfokuskan pada seleksi kelas bahaya spesifik serta menyusun ulang data menjadi 5 *fold* eksperimen. Sebagai langkah antisipasi terhadap keterbatasan data setelah seleksi kelas, strategi augmentasi temporal diterapkan untuk memperkaya variasi data latih. Oleh karena itu, tahap prapemrosesan data menjamin integritas data dan mencegah kebocoran informasi (*data leakage*) selama proses pelatihan, sebagaimana dijabarkan pada tahapan berikut.

3.2.1 Seleksi Kelas

Tidak seluruh kelas pada *dataset* UrbanSound8K relevan dengan konteks keselamatan tunarungu. Pemilihan kelas suara ini didasari oleh tingkat urgensi ancaman dan karakteristik suara yang menuntut respons fisik seketika (*immediate physical response*) bagi penyandang tunarungu di lingkungan luar ruangan [46]. Oleh karena itu, dilakukan penyaringan untuk hanya mengambil 4 kelas prioritas yang merepresentasikan sinyal bahaya, yaitu:

1. ***gun_shot* (Tembakan)**

   Suara tembakan senjata api merupakan sinyal bahaya dengan tingkat fatalitas yang sangat tinggi. Jika penyandang tunarungu tidak menyadari suara tersebut dan mengambil langkah antisipasi secepatnya, maka mereka berpotensi terkena tembakan senjata api yang dapat melukai bahkan hingga mengancam keselamatan jiwa.

2. ***siren* (Sirine)**

   Suara sirine menandakan keberadaan kendaraan prioritas (ambulans, pemadam kebakaran, atau patroli polisi) yang sering kali melaju dengan kecepatan tinggi dan memiliki hak prioritas jalan. Dengan deteksi suara tersebut, pengguna dapat mengambil langkah untuk segera menepi dan menghindari jalur lintasan kendaraan tersebut.

3. ***dog_bark* (Gonggongan Anjing)**

   Suara anjing yang menggonggong berpotensi menjadi sinyal bahaya, terutama jika anjing tersebut berupa anjing liar yang dapat menyerang secara tiba-tiba. Dengan mendeteksi suara ini, pengguna dapat langsung mengetahui di mana posisi anjing tersebut dan mengambil langkah untuk antisipasi ancaman.

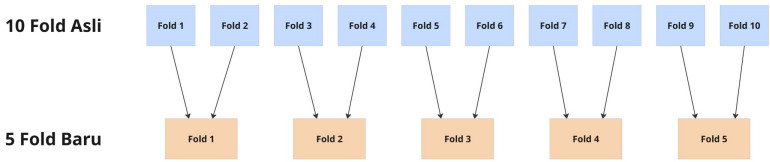
4. ***car_horn* (Klakson Mobil)**

   Suara klakson mobil menjadi sinyal penting pada lalu lintas, yang salah satu fungsinya adalah sebagai peringatan dari pengemudi kendaraan kepada pengguna jalan lainnya yang berada di jalur lintasnya. Secara statistik, penyandang tunarungu memiliki risiko terlibat kecelakaan lalu lintas 70% lebih tinggi karena hambatan dalam menerima isyarat suara dari lingkungan sekitarnya, seperti suara klakson kendaraan [47]. Dengan deteksi ini, pengguna dapat lebih waspada terhadap situasi lalu lintas di sekitarnya.

Sebaliknya, kelas suara yang berpotensi menandakan bahaya seperti mesin bor (*drilling*) dan pemecah beton (*jackhammer*) tidak diikutsertakan. Hal ini dikarenakan kedua sumber suara tersebut lebih bersifat sebagai gangguan berkelanjutan (*continuous ambient noise*) yang relevan untuk aktivitas kontruksi dan posisinya bersifat statis. Karakteristik suara tersebut tidak menuntut respons menghindar yang mendadak, sehingga kurang relevan dengan skenario keselamatan mobilitas darurat yang menjadi batasan pada penelitian ini.

3.2.2 Strategi Penyusunan Ulang Data (*Fold Mapping*)

Dataset UrbanSound8K secara bawaan terbagi ke dalam 10 *fold*. Untuk efisiensi eksperimen tanpa melanggar aturan independensi data, penelitian ini menyusun ulang 10 *fold* tersebut menjadi 5 *fold* eksperimen baru. Penyusunan ini dilakukan secara berurutan (misalnya *fold* 1 dan 2 menjadi *fold* baru 1) tanpa pengacakan data antar *fold*. Teknik penyusunan tersebut memastikan data latih dan data uji tidak tercampur pada *fold* yang sama, sehingga menghindari terjadinya kebocoran data (*data leakage*) yang menyebabkan model menjadi *overfitting*.



Gambar 3.2 Ilustrasi Strategi Penyusunan Ulang *Fold* pada *Dataset* UrbanSound8K

Adapun penyusunan ini didasari oleh dua pertimbangan, yaitu rasio pembagian data yang ideal dan efisiensi sumber daya komputasi. Skema 5-*fold* menghasilkan proporsi 80% data latih dan 20% data uji. Proporsi ini memberikan model evaluasi yang lebih representatif terhadap variasi data daripada skema 10-*fold* bawaan yang menyisakan 10% data uji. Selain memberikan porsi data yang ideal, skema tersebut berdampak mengurangi waktu komputasi pelatihan model tanpa mengurangi validitas pengujian.

Dikarenakan skema baru tersebut, maka mekanisme validasi yang digunakan selanjutnya pada model adalah *5-fold cross validation*. Pada iterasi setiap pengujiannya, satu *fold* dialokasikan sebagai data uji secara bergantian dan empat *fold* sisanya akan digunakan sebagai data latih model. Mekanisme

rotasi ini diterapkan agar model selalu diuji dengan data yang berbeda dari yang sudah pernah dipelajari saat proses latih, sehingga hasil evaluasi mencerminkan kemampuan generalisasi yang sebenarnya.

3.2.3 Standarisasi Format Audio

Sebelum data audio dapat diproses oleh model, diperlukan standarisasi format pada data tersebut mengingat data mentah memiliki format yang berbeda-beda. Ketidaksesuaian format *input* dapat menyebabkan kegagalan pada proses ekstraksi fitur, bahkan pelatihan model. Oleh karena itu, standarisasi format menjadi langkah penting untuk memastikan bahwa semua data audio memiliki format yang konsisten dan sesuai dengan kebutuhan model, yang mencakup :

1. **Konversi *Channel* Audio (*Down-Mixing*)**

   Langkah ini diterapkan karena terdapat beberapa data audio pada *Dataset* UrbanSound8K dengan format *stereo* (dua saluran). Untuk menyeragamkan dimensi *input* pada model nanti, seluruh data audio dikonversi menjadi format *mono* (satu saluran). Proses *down-mixing* ini menggunakan persamaan **Rumus 2.1** pada Bab II.

   Konversi ini dilakukan untuk menyederhanakan representasi suara untuk keperluan klasifikasi dan mengurangi kompleksitas komputasi.

2. **Penyesuaian *Sampling Rate* (*Resampling*)**

   Setelah audio diubah menjadi format *mono*, dilakukan penyesuaian *sampling rate* (*resampling*). *Sampling rate* pada *dataset* UrbanSound8K bervariatif, sehingga *sampling rate* pada data diubah menjadi 32.000 Hz agar sesuai dengan standar *input* pada model PANNs.

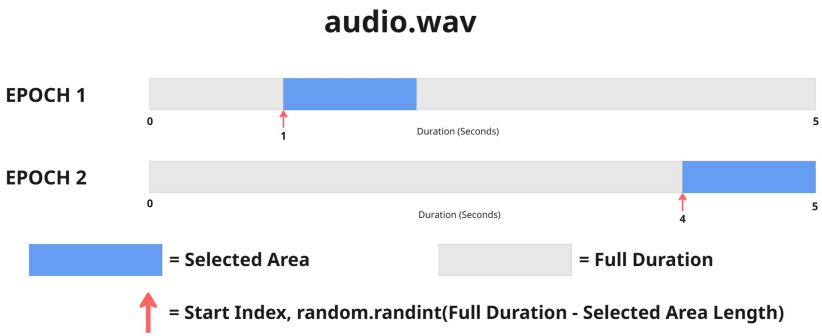
3.2.4 Augmentasi Temporal (*Cropping*)

Seleksi kelas bahaya yang telah dilakukan menghasilkan jumlah data yang relatif sedikit pada kelas yang terpilih, yang berpotensi membuat model kekurangan data untuk latih dan uji. Oleh karena itu, teknik augmentasi hadir

sebagai solusi untuk memperkaya variasi data tanpa perlu menambah jumlah sampel dengan *dataset* lain. Teknik ini juga mempersingkat waktu dan sumber daya komputasi dikarenakan model hanya mempelajari sebagian fitur penting dari seluruh bagian fitur pada data aslinya.

Sebelum melakukan proses augmentasi, durasi audio akan diperiksa terlebih dahulu apakah sesuai dengan standar yang telah ditetapkan, yaitu 5 detik. Durasi 5 detik ini dipilih karena *dataset* UrbanSound8K memiliki data audio yang berdurasi rata-rata 4 detik, sehingga 5 detik merupakan durasi yang cukup aman untuk dijadikan standar dalam penelitian ini. Jika audio asli berdurasi kurang dari 5 detik, maka audio akan ditambahkan *zero-padding* hingga durasinya mencapai 5 detik. Jika audio asli berdurasi lebih dari 5 detik, maka audio akan dipotong hingga berdurasi 5 detik. Penetapan standar 5 detik durasi audio merupakan protokol keamanan *pipeline* untuk antisipasi terhadap inkonsistensi durasi akibat anomali data, serta memberikan ruang jelajah yang luas pada algoritma augmentasi. Setelah standarisasi durasi selesai, maka data siap untuk diproses menggunakan teknik augmentasi.

Teknik augmentasi yang diterapkan adalah *random cropping*, yaitu pemotongan acak pada segmen audio sepanjang durasi tertentu. Pada setiap *epoch* pelatihan, segmen 1 detik akan diambil secara acak dari durasi 5 detik tersebut. Ini memberikan variasi temporal pada model untuk belajar mengenali karakteristik suara yang berbeda dari berbagai posisi dalam rekaman asli pada setiap *epoch*. Strategi ini memastikan bahwa data uji memiliki dimensi yang konsisten, sehingga performa model dapat dievaluasi secara stabil dan tolok ukur pengujian dapat direproduksi dengan akurat.



Gambar 3.3 Ilustrasi Teknik *Random Cropping*

Teknik *random cropping* ini tidak diterapkan pada data uji dikarenakan berpotensi mengubah distribusi data uji, sehingga model tidak dapat mencapai nilai evaluasi secara maksimal. Agar tetap menjaga konsistensi performa uji model, diterapkan *center cropping* pada data uji dengan mengambil segmen audio sepanjang 1 detik, dimulai dari setengah durasi maksimal audio. Dalam penelitian ini, pada audio dengan durasi maksimal 5 detik, ekstraksi dilakukan dengan menjadikan detik ke-2,5 sebagai titik pusat (*center point*), sehingga model memproses segmen audio yang berada pada rentang detik ke-2 hingga detik ke-3.

3.3 Konfigurasi Model

Model yang digunakan dalam penelitian ini adalah model PANNs [7] dengan 3 arsitektur berbeda, yaitu *Res1dNet31*, *ResNet38*, dan *Wavegram-Logmel-CNN*. Ketiga arsitektur ini telah terbukti memiliki performa yang baik pada tugas klasifikasi audio berdasarkan masing-masing representasi fiturnya.

Dalam konteks penelitian ini, pemilihan ketiga arsitektur tersebut didasarkan pada perbedaan karakteristik representasi data audio yang digunakan, yaitu *raw waveform*, *log-mel spectrogram*, dan pendekatan *hybrid*. Setiap representasi memiliki sifat yang berbeda dalam menangkap informasi sinyal, di mana *raw waveform* mempertahankan informasi temporal secara utuh, sedangkan

log-mel spectrogram menekankan struktur pola frekuensi yang lebih teratur. Berdasarkan perbedaan tersebut, penelitian ini mengajukan hipotesis bahwa pendekatan *hybrid* yang menggabungkan karakteristik komplementer dari kedua representasi berpotensi memberikan performa yang lebih optimal dalam tugas *Environmental Sound Classification* (ESC), meskipun efektivitasnya tetap dipengaruhi oleh kompleksitas model dan keterbatasan data. Hipotesis ini selanjutnya akan diuji melalui perbandingan performa ketiga arsitektur yang digunakan dalam penelitian ini.

Namun, agar model dapat berfungsi optimal sesuai dengan konteks klasifikasi 4 kelas bahaya pada *dataset* UrbanSound8K, diperlukan beberapa penyesuaian konfigurasi model, yaitu sebagai berikut.

3.3.1 Modifikasi Arsitektur PANNs

Modifikasi beberapa komponen pada arsitektur PANNs diperlukan pada lapisan keluaran (*output layer*). Langkah ini dilakukan mengingat jumlah kelas pada *output layer* yang belum sesuai dengan target kelas suara yang ditentukan dan penyesuaian jenis *output* klasifikasi PANNs yang awalnya *multi-label*. Oleh karena itu, dilakukan dua penyesuaian utama:

1. **Penyesuaian Jumlah *Neuron* pada Lapisan Akhir**

   Jumlah *neuron* pada lapisan akhir disesuaikan dari 527 *neuron* menjadi 4 *neuron* (sesuai jumlah kelas pilihan).

2. **Penyesuaian Fungsi Aktivasi pada Lapisan Akhir**

   Pada model asli PANNs, lapisan akhir menggunakan fungsi aktivasi *sigmoid* yang mengeluarkan output berupa probabilitas tiap kelasnya sendiri (cocok untuk *multi-label*). Namun, klasifikasi model yang ditentukan pada penelitian ini adalah klasifikasi *single-label*, sehingga terjadi modifikasi pada lapisan akhir pada arsitektur model menjadi lapisan *linear* yang mengeluarkan angka mentah (*logits*). Nilai *logits* ini nantinya akan dikonversi menjadi distribusi probabilitas tunggal oleh

fungsi `nn.CrossEntropyLoss()` yang telah mengandung fungsi *softmax* selama fase pelatihan. Dengan perubahan ini, model akan memilih satu kelas pemenang dengan probabilitas tertinggi daripada kelas lainnya.

3.3.2 Parameter Konfigurasi *Input*

Selain melakukan modifikasi arsitektur, konfigurasi parameter *input* juga disesuaikan dengan standar yang ditetapkan pada PANNs untuk menjamin kompatibilitas dimensi fitur antara data *input* dengan bobot *pre-trained*, sehingga ekstraksi fitur dapat berjalan optimal tanpa adanya kesalahan dimensi. Rincian konfigurasi tersebut meliputi:

Tabel 3.1 Konfigurasi Parameter *Input* Audio Model PANNs

Parameter	**Nilai**	**Keterangan**
Sampling Rate	32.000 Hz	Frekuensi pencuplikan sinyal audio
Window Size	1024	Ukuran jendela FFT (*fast Fourier transform*)
Hop Size	320	Jarak pergeseran antar jendela waktu
Mel-bins	64	Jumlah *filter bank* pada skala Mel (dimensi fitur vertikal)
F-min	50 Hz	Batas frekuensi minimum
F-max	14.000 Hz	Batas frekuensi maksimum

Penggunaan *sampling rate* 32 kHz dipilih merujuk pada spesifikasi standar PANNs [7]. Angka ini juga telah memenuhi kriteria teorema Nyquist-Shannon untuk menangkap frekuensi hingga 16 kHz. Rentang ini dinilai efisien karena sebagian besar energi spektral pada suara bahaya (seperti sirine, klakson, dan tembakan) berada pada rentang 50 Hz hingga 14.000 Hz, sehingga parameter ini dapat mempertahankan informasi vital sekaligus mereduksi beban komputasi dibandingkan standar audio 44.1 kHz.

3.4 Eksperimen *Transfer Learning*

Konsep utama dari *transfer learning* adalah memanfaatkan pengetahuan yang telah dipelajari oleh model terlatih sebelumnya (*pre-trained*) dengan *dataset* berskala besar untuk diterapkan pada permasalahan spesifik yang memiliki

keterbatasan data. Dalam penelitian ini, arsitektur PANNs yang digunakan merupakan *pre-trained model* yang sebelumnya telah dilatih menggunakan *dataset* AudioSet [7]. AudioSet merupakan sebuah *dataset* audio berskala besar yang berisi lebih dari 2.084.320 klip suara berdurasi 10 detik yang mencakup berbagai macam suara manusia, hewan, alat musik, genre dan suara lingkungan dengan 632 kategori kelas (*multi-label classification*) [8]. Adapun data yang digunakan pada saat melatih model PANNs berjumlah 1,9 juta klip suara yang mencakup 527 kategori kelas dari AudioSet [7].

Pada fase *pre-training* tersebut, para peneliti PANNs melatih beberapa varian arsitekturnya menggunakan pendekatan representasi fitur yang spesifik, yakni *raw waveform*, *log-mel spectrogram*, serta kombinasi keduanya (*hybrid*), untuk mengenali berbagai pola hierarkis suara [7]. Pengetahuan berupa parameter bobot ekstraktor fitur dari masing-masing varian arsitektur inilah yang nantinya akan diadaptasikan ke dalam domain penelitian ini.

Setelah model dikonfigurasi sesuai kebutuhan penelitian, tahap selanjutnya adalah melakukan *fine-tuning* model secara menyeluruh pada *dataset* UrbanSound8K. Tahapan ini melibatkan konfigurasi *hyperparameter* pelatihan secara eksperimental dan penerapan beberapa strategi pelatihan yang bertujuan untuk mendapatkan hasil evaluasi yang optimal. Beberapa strategi teknis yang diterapkan mencakup pengondisian lingkungan implementasi, penerapan standar reproduksibilitas, penanganan ketidakseimbangan data, serta mekanisme kontrol pelatihan untuk memastikan model mempelajari data dengan baik dan menghindari *overfitting*. Penggunaan Weights & Biases (WandB) sebagai alat pemantauan dan pencatatan eksperimen pelatihan model secara *real-time* termasuk ke dalam strategi untuk memastikan transparansi dan menjaga validitas dalam merekam hasil eksperimen. Rincian beberapa langkah strategi teknis tersebut dijabarkan sebagai berikut.

3.4.1 Lingkungan Implementasi dan Reproduksibilitas

Seluruh rangkaian eksperimen pelatihan dijalankan pada satu perangkat komputasi terdedikasi guna menjaga konsistensi performa dan waktu komputasi. Mengingat perangkat keras memiliki dampak signifikan terhadap efisiensi proses *fine-tuning* model, rincian spesifikasi perangkat keras (*hardware*) yang digunakan dijabarkan pada Tabel 3.2.

Tabel 3.2 Spesifikasi Perangkat Keras untuk Pelatihan

Komponen	**Spesifikasi**	**Keterangan**
Processor (CPU)	Intel Core i7-7700HQ	Bertanggung jawab atas *pre-processing* data, augmentasi dan manajemen *dataloader*.
Kartu Grafis (GPU)	NVIDIA GeForce GTX 1050 (4 GB VRAM)	Berperan dalam akselerasi komputasi *tensor* selama pelatihan model.
Memori (RAM)	8 GB DDR4	Menunjang alokasi data sementara dengan kecepatan transfer 2400 MHz.
Penyimpanan	1 TB HDD	Digunakan sebagai repositori penyimpanan *dataset* utama.

Selain perangkat keras, lingkungan perangkat lunak juga dikonfigurasi menggunakan pustaka (*library*) standar industri untuk menjamin reliabilitas eksperimen pelatihan. Implementasi dilakukan menggunakan bahasa pemrograman Python dengan kerangka kerja utama PyTorch. Rincian lengkap *library* yang digunakan beserta fungsinya ditampilkan pada Tabel 3.3.

Tabel 3.3 Daftar Perangkat Lunak dan Pustaka Pendukung

Perangkat Lunak	**Fungsi Utama**
Python	Bahasa pemrograman dasar untuk seluruh implementasi teknis penelitian.
PyTorch	Kerangka kerja utama untuk pembangunan, pelatihan, dan evaluasi model *deep learning*.
NumPy	Pustaka komputasi numerik untuk manipulasi matriks dan *array*.
Scikit-learn	Pustaka untuk perhitungan metrik evaluasi (*F1-score*) dan *confusion matrix*.
Matplotlib & Seaborn	Pustaka visualisasi data untuk pembuatan grafik hasil pelatihan.
Weights & Biases	Platform pemantauan (*monitoring*) metrik pelatihan secara *real-time*.

Guna memastikan eksperimen dapat direproduksi ulang (*reproducible*) pada aspek logika program, dilakukan penguncian benih acak (*random seed*) bernilai tetap (42) pada tiga komponen pustaka utama, yaitu:

1. **Python Native Random**

   Mengunci pengacakan pada operasi dasar.

2. **NumPy**

   Menjamin konsistensi pada operasi manipulasi matriks dan pembagian data (*fold splitting*).

3. **PyTorch (CPU & CUDA)**

   Mengunci inisialisasi bobot awal jaringan saraf agar start pelatihan selalu sama.

Meskipun inisialisasi bobot dan pembagian data telah dikunci secara deterministik, konfigurasi `cudnn.deterministic` pada *backend* GPU tetap dinonaktifkan (*False*). Keputusan ini diambil untuk memprioritaskan efisiensi sumber daya dan kecepatan komputasi, mengingat penegakan determinisme penuh pada operasi konvolusi GPU dapat meningkatkan waktu pelatihan secara signifikan. Variasi hasil numerik mikroskopis yang mungkin timbul akibat optimasi perangkat keras ini dinilai dapat diabaikan (*negligible*) dan tidak

mempengaruhi validitas performa model secara keseluruhan.

3.4.2 Penanganan Ketidakseimbangan Data (*Cost-Sensitive Learning*)

Meskipun *dataset* UrbanSound8K memiliki distribusi data yang relatif terstruktur, proses seleksi kelas dan penyusunan ulang *fold* berpotensi menghasilkan ketidakseimbangan jumlah sampel antar kelas. Ketidakseimbangan ini dapat memicu bias pada model, di mana model akan cenderung memprediksi kelas mayoritas (jumlah sampel banyak) dan mengabaikan kelas minoritas karena kontribusinya terhadap nilai *loss* yang kecil.

Untuk memitigasi risiko tersebut tanpa perlu membuang data latih (*undersampling*), diterapkan pendekatan *cost-sensitive learning* [38]. Sesuai dengan landasan teori pada Bab II, strategi ini diterapkan dengan memberikan bobot penalti (*class weights*) yang dikalkulasikan menggunakan Rumus 2.4. Pada tahap implementasi teknis, nilai bobot penalti ini dihitung secara dinamis pada setiap iterasi *fold* menggunakan fungsi `compute_class_weight` dari pustaka Scikit-learn. Hasil nilai bobot dari algoritma tersebut kemudian dikonversi menjadi *tensor* satu dimensi dan dimasukkan langsung sebagai parameter `weight` ke dalam fungsi `nn.CrossEntropyLoss()` milik PyTorch.

Sebagai ilustrasi praktis, kelas *gun_shot* memiliki jumlah sampel yang jauh lebih sedikit dibandingkan kelas *car_horn*. Berdasarkan kalkulasi algoritma pembobotan, kelas *gun_shot* secara otomatis memiliki nilai bobot pengali (w) yang jauh lebih besar. Apabila model melakukan kesalahan prediksi pada sampel *gun_shot*, nilai *loss* yang dihasilkan akan dikalikan dengan bobot yang besar tersebut. Mekanisme hukuman numerik yang lebih berat ini memaksa arsitektur model untuk memberikan prioritas pembelajaran yang setara pada fitur kelas minoritas, terlepas dari seberapa minim jumlah sampelnya di dalam *batch* pelatihan.

3.4.3 Konfigurasi *Optimizer* dan *Scheduler*

Proses *fine-tuning* dilakukan secara menyeluruh, baik pada lapisan *backbone* maupun *head* tanpa melakukan pembekuan bobot (*unfrozen layers*). Hal ini bertujuan agar fitur ekstraktor pada model PANNs dapat beradaptasi sepenuhnya terhadap karakteristik fitur yang diambil dari audio *dataset* UrbanSound8K, khususnya pada 4 kelas yang telah diseleksi.

Pembaruan bobot model (*weight update*) dilakukan menggunakan *optimizer* AdamW (*Adam with decoupled weight decay*). Pemilihan AdamW didasari oleh kemampuannya dalam memisahkan mekanisme pembaruan bobot dengan *weight decay*, sehingga lebih efektif dalam mencegah *overfitting* dibandingkan algoritma Adam standar. Pelatihan dilakukan dalam *batch size* sebesar 8 selama maksimal 50 *epoch*, dengan pengaturan *learning rate* awal sebesar 5e-4 dan nilai *weight decay* sebesar 1e-4.

Untuk mengoptimalkan proses adaptasi pelatihan model, diterapkan pula penyesuaian laju pembelajaran dinamis menggunakan *scheduler* ReduceLROnPlateau. Mekanisme ini dirancang untuk memantau metrik *validation loss*. Apabila nilai kerugian validasi tersebut tidak mengalami penurunan selama 3 *epoch* berturut-turut (*patience*), maka nilai *learning rate* akan direduksi secara otomatis sebesar setengah dari nilai sebelumnya (faktor 0,5). Strategi ini membantu model untuk menemukan detail kecil global dengan lebih presisi (konvergensi) saat mendekati akhir pelatihan.

3.4.4 Strategi Optimasi (*Early Stopping* dan *Checkpoint*)

Mengingat pelatihan dilakukan pada *dataset* terbatas, risiko *overfitting* pada *epoch* lanjutan menjadi sangat tinggi. Untuk mencegah hal tersebut, diterapkan dua mekanisme pencegahan otomatis:

1. ***Early Stopping***

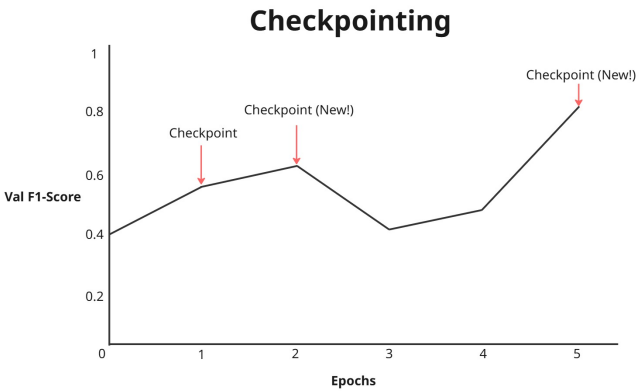
   Mekanisme *Early Stopping* berfungsi sebagai penghenti darurat dalam pelatihan model. Hal ini dilakukan untuk mencegah model yang sudah

mencapai performa maksimal lebih awal menjadi *overfitting* pada data latih.

Secara teknis, meskipun batas maksimal jumlah *epoch* telah ditetapkan sebanyak 50 *epoch*, durasi sebenarnya bergantung pada pemantauan metrik *Validation loss* sebagai tolok ukur performa model. Ketika metrik tersebut tidak mengalami penurunan melebihi rekor minimal selama 5 *epoch* berturut-turut (*patience*), pelatihan model akan dihentikan secara paksa lebih awal. Mekanisme ini menyebabkan model tidak diberikan kesempatan untuk terus belajar data yang nantinya akan dihafal oleh model tersebut.

2. ***Model Checkpointing***

   Mekanisme *checkpointing* berfungsi untuk menyimpan parameter bobot model terbaik secara otomatis. Selama proses pelatihan berjalan, sistem secara dinamis hanya akan menyimpan *file* bobot pada *epoch* yang berhasil mencetak nilai *F1-score* validasi tertinggi. Jika performa pada *epoch* selanjutnya memburuk, parameter bobot model tidak akan diperbarui, sehingga hasil akhir yang dievaluasi dipastikan adalah versi model dengan konfigurasi bobot paling optimal.



Gambar 3.4 Ilustrasi Mekanisme *Checkpointing*

Contoh mekanisme ini dapat dilihat pada Gambar 3.4. Pada *epoch* ke-1 dan ke-2, model mencetak nilai *validation F1-score* yang terus meningkat, sehingga *checkpointing* dilakukan 2 kali berturut-turut (memperbarui rekor lama). Namun pada *epoch* ke-3, nilai tersebut menurun cukup drastis, sehingga sistem tetap mempertahankan bobot dari *epoch* ke-2. Walaupun terdapat kenaikan nilai pada *epoch* ke-4, capaiannya belum melampaui rekor *epoch* ke-2, sehingga tidak terjadi *checkpointing* baru. Pembaruan *checkpoint* akhirnya dilakukan kembali pada *epoch* ke-5 karena nilainya berhasil mengalahkan rekor *epoch* ke-2. Dengan demikian, parameter bobot dari *epoch* ke-5 inilah yang disimpan sebagai model final untuk tahap pengujian.

3.4.5 Pemantauan dan Pencatatan Eksperimen

Dalam memantau pergerakan metrik evaluasi pelatihan (mencakup *train/validation loss*, *F1-score*, dan pergerakan *learning rate*), metrik tersebut direkam dan divisualisasikan secara *real-time* menggunakan alat bantu berupa *dashboard* Weights & Biases (WandB). Penggunaan alat ini memastikan transparansi rekam jejak pelatihan model. Di akhir setiap pengujian *fold*,

sistem juga secara otomatis menggenerasi laporan grafik riwayat pelatihan dengan menampilkan metrik yang dijadikan bahan evaluasi.

3.4.6 Konfigurasi Parameter Pelatihan

Rincian skenario pelatihan tadi telah membawa beberapa *hyperparameter* yang akan dijadikan sebagai tolok ukur eksperimen pada pelatihan model. *Hyperparameter* yang dikendalikan dalam eksperimen ini disajikan pada Tabel 3.4. Nilai-nilai yang ditetapkan dalam tabel ini ditetapkan berdasarkan studi literatur dan eksperimen awal untuk mendapatkan konvergensi yang optimal.

Tabel 3.4 Parameter Konfigurasi Pelatihan

Kategori	**Konfigurasi / Nilai**
Hyperparameter Training	
Batch Size	8
Epoch	50
Num Workers	2
Learning Rate Awal	0.0005 ($5e^{-4}$)
Optimizer	
Tipe	AdamW
Weight Decay	0.0001 ($1e^{-4}$)
Learning Rate Scheduler	
Tipe	ReduceLROnPlateau
Faktor Pengurangan	0.5
Patience	3
Target Metrik	Validation Loss
Early Stopping	
Patience	5
Target Metrik	Validation F1-score <= Best Validation F1-score

3.5 Analisis dan Evaluasi

Setelah proses pelatihan selesai, kinerja model diukur menggunakan tiga indikator evaluasi utama, yaitu:

1. ***Macro F1-score***

   Metrik ini menjadi tolok ukur utama untuk mengevaluasi akurasi dan sensitivitas model secara harmonis. Penggunaan *macro F1-score* memastikan setiap kelas diprioritaskan secara setara tanpa dipengaruhi oleh ketidakseimbangan jumlah data (*imbalanced data*). Dikarenakan mekanisme pengujian menggunakan *5-fold cross validation*, maka hasil akhir *macro F1-score* pada setiap model dengan representasi *input* tertentu akan dilaporkan dalam format rata-rata beserta rentang simpangannya (Rata-rata ± Standar Deviasi).

2. **Kurva *Loss* (*Loss Curve*)**

   Metrik ini digunakan untuk memantau dinamika proses pembelajaran model guna mendeteksi indikasi *overfitting* atau *underfitting* selama proses pelatihan berlangsung.

3. ***Confusion Matrix***

   Matriks ini digunakan untuk memetakan detail distribusi prediksi kelas yang benar dan keliru secara spesifik pada masing-masing kelas target.

Selain menggunakan ketiga indikator tersebut, penelitian ini juga menerapkan uji statistik formal untuk membandingkan performa akhir dari setiap model dengan ketiga representasi *input* (*raw waveform*, *log-mel spectrogram*, dan *hybrid*). Uji statistik yang digunakan adalah *Wilcoxon signed-rank test*, yaitu sebuah uji non-parametrik yang beroperasi pada hasil evaluasi kelima (*fold*) dari masing-masing model. Penerapan uji ini bertujuan untuk membuktikan secara matematis apakah selisih keunggulan performa antar model benar-benar signifikan secara ilmiah (dengan ambang batas probabilitas $p < 0.05$) atau hanya terjadi akibat variansi data acak.

DAFTAR PUSTAKA

[1] World Health Organization. *Global Status Report on Road Safety 2023*. Geneva: World Health Organization, 2023. ISBN: 9789240086517.

[2] Birgitta Thorslund et al. "Effects of hearing loss on traffic safety and mobility". *European Transport Research Review* 5 (2013), pp. 113–121.

[3] Brian C. J. Moore. "Perceptual Consequences of Cochlear Hearing Loss and their Implications for the Design of Hearing Aids". *Ear and Hearing* 17.2 (1996), pp. 133–161.

[4] Michael Büchler et al. "Sound classification in hearing aids inspired by auditory scene analysis". *The Journal of the Acoustical Society of America* 118.3 (2005), pp. 2057–2057.

[5] Justin Salamon, Christopher Jacoby, and Juan Pablo Bello. "A Dataset and Taxonomy for Urban Sound Research". *Proceedings of the 22nd ACM International Conference on Multimedia*. ACM. 2014, pp. 1041–1044.

[6] Karol J Piczak. "Environmental sound classification with convolutional neural networks". *2015 IEEE 25th International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE. 2015, pp. 1–6.

[7] Qiuqiang Kong et al. "PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition". *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 28 (2020), pp. 2880–2894.

[8] Jort F Gemmeke et al. "Audio set: An ontology and human-labeled dataset for audio events". *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2017, pp. 776–780.

[9] Justin Salamon and Juan Pablo Bello. "Deep Convolutional Neural Networks and Data Augmentation for Environmental

Sound Classification". *IEEE Signal Processing Letters* 24.3 (2017), pp. 279–283.

[10] Muhammad Huzaifah. "Comparison of time-frequency representations for environmental sound classification using convolutional neural networks". *arXiv preprint arXiv:1706.07156* (2017).

[11] Yuji Tokozume and Tatsuya Harada. "Learning environmental sounds with end-to-end convolutional neural network". *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2017, pp. 2721–2725.

[12] Wei Dai et al. "Very deep convolutional neural networks for raw waveforms". *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2017, pp. 421–425.

[13] A. Avadhani et al. "Multi-Class Urban Sound Classification with Deep Learning Architectures". *Journal of Environmental Acoustics* (2024).

[14] Tasmiya Mujawar et al. "Urban Noise Classification Using Machine Learning Techniques: Comparative Analysis and Future". *International Journal of Machine Learning and Computing* (2025).

[15] W. Nogueira et al. "Transformers for Urban Sound Classification—A Comprehensive Performance Evaluation". *Applied Sciences* (2022).

[16] R. Trivedi et al. "Deep Learning for Urban Sound Classification: Using CNN and YAMNet Model Integration". *IEEE Access* (2025).

[17] Yu Shen. "A Study on AI-Driven Acoustic Feature Design for Urban Noise Classification". PhD thesis. Ulsan, South Korea: University of Ulsan, 2025.

[18] Anam Bansal and Naresh Kumar Garg. "Environmental Sound Classification: A descriptive review of the literature". *Intelligent Systems with Applications* 16 (2022), p. 200115.

[19] Sebastien Tremblay et al. “Exploiting Environmental Sounds for Activity Recognition in Smart Homes”. *Artificial Intelligence Applied to Assistive Technologies and Smart Environments* (2015).

[20] Dongping Zhang et al. “An Automatic Classification System for Environmental Sound in Smart Cities”. *Sensors* 23.15 (2023), p. 6823.

[21] Alan V Oppenheim and Ronald W Schafer. *Discrete-Time Signal Processing*. 3rd. Upper Saddle River, NJ: Pearson, 2010.

[22] Ze-Nian Li, Mark S Drew, and Jiangchuan Liu. *Fundamentals of Multimedia*. 2nd. Cham: Springer, 2014.

[23] David A Patterson and John L Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. 5th. Waltham, MA: Morgan Kaufmann, 2013.

[24] Jiawei Han, Jian Pei, and Micheline Kamber. *Data mining: concepts and techniques*. Elsevier, 2011.

[25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[26] John G Proakis and Dimitris G Manolakis. *Digital signal processing: principles, algorithms, and applications*. Pearson Prentice Hall, 2006.

[27] Via Wahyuningtyas. “Implementasi Ekstraksi Fitur untuk Klasifikasi Suara Urban Menggunakan Deep Learning”. *Jurnal SAKTI (Sains, Aplikasi, Komputasi dan Teknologi Informasi)* 3.1 (2021). Diakses pada April 2026, pp. 10–17.

[28] Meinard Müller. *Fundamentals of music processing: Audio, analysis, algorithms, applications*. Springer, 2015.

[29] Kaiming He et al. “Deep residual learning for image recognition”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[30] Stanley Smith Stevens, John Volkmann, and Edwin B Newman. “A scale for the measurement of the psychological magnitude pitch”. *The Journal of the Acoustical Society of America* 8.3 (1937), pp. 185–190.

[31] Hendrik Purwins et al. “Deep learning for audio signal processing”. *IEEE Journal of Selected Topics in Signal Processing* 13.2 (2019), pp. 206–219.

[32] Sinno Jialin Pan and Qiang Yang. “A survey on transfer learning”. *IEEE Transactions on knowledge and data engineering* 22.10 (2009), pp. 1345–1359.

[33] Jason Yosinski et al. “How transferable are features in deep neural networks?” *Advances in neural information processing systems*. 2014, pp. 3320–3328.

[34] James Kirkpatrick et al. “Overcoming catastrophic forgetting in neural networks”. *Proceedings of the national academy of sciences* 114.13 (2017), pp. 3521–3526.

[35] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. *Neural networks: Tricks of the trade*. Springer, 2012, pp. 437–478.

[36] Adam Paszke et al. “PyTorch: An imperative style, high-performance deep learning library”. *Advances in neural information processing systems*. 2019, pp. 8026–8037.

[37] Justin M Johnson and Taghi M Khoshgoftaar. “Survey on deep learning with class imbalance”. *Journal of Big Data* 6.1 (2019), pp. 1–54.

[38] Charles Elkan. “The Foundations of Cost-Sensitive Learning”. *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*. Vol. 17. 1. Seattle, WA, USA: Morgan Kaufmann Publishers Inc., 2001, pp. 973–978.

[39] Fabian Pedregosa et al. “Scikit-learn: Machine learning in Python”. *the Journal of machine Learning research* 12 (2011), pp. 2825–2830.

[40] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. *arXiv preprint arXiv:1412.6980* (2014).

[41] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. *International Conference on Learning Representations*. 2019.

[42] Lutz Prechelt. “Early stopping-but when?” *Neural Networks: Tricks of the trade*. Springer, 1998, pp. 55–69.

[43] Tom Fawcett. “An introduction to ROC analysis”. *Pattern recognition letters* 27.8 (2006), pp. 861–874.

[44] David MW Powers. “Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation”. *Journal of Machine Learning Technologies* 2.1 (2011), pp. 37–63.

[45] Janez Demšar. “Statistical comparisons of classifiers over multiple data sets”. *Journal of Machine learning research* 7.Jan (2006), pp. 1–30.

[46] Dhruv Jain et al. “SoundWatch: Exploring smartwatch-based deep learning approaches to support sound awareness for deaf and hard of hearing users”. *The 22nd International ACM SIGACCESS Conference on Computers and Accessibility*. 2020, pp. 1–13.

[47] L. Junghee. “A.Ds: Aid Device for Deaf Drivers”. *12th Latin American and Caribbean Conference for Engineering and Technology (LACCEI)*. Guayaquil, Ecuador, 2014.